



復旦大學
FUDAN UNIVERSITY

算法设计与分析（第二讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

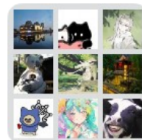
本人简介

- 计算与智能创新学院，青年研究员
- 研究方向：人工智能安全，数据隐私保护
- 办公室：复旦大学江湾校区二号交叉学科楼 D4027
- 个人主页：chenghuang-sec.github.io

助教

- 陈依琳, 25113050211@m.fudan.edu.cn
 - 朱钊毅, 26213050522@m.fudan.edu.cn
 - 罗瑞, 26113050220@m.fudan.edu.cn
 - 王意飞, 26213050373@m.fudan.edu.cn
-
- 助教答疑:
 - 微信/邮件: 早9点到晚9点, 工作日
 - 面对面交流: 周五(双)晚, 习题课 课间/结束后

课程微信群



群聊：算法26课程群



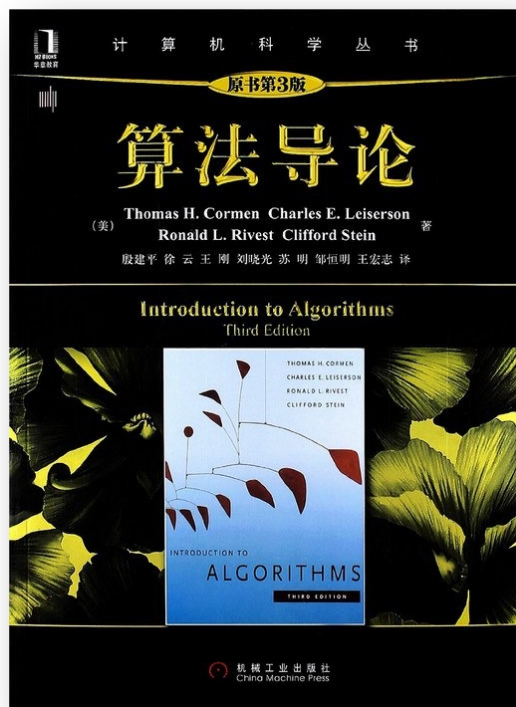
该二维码7天内(9月23日前)有效，重新进入将更新

考核

- 平时成绩（30%）
 - 考勤（10%）（不点名，有事请假）
 - 平时作业（20%，通过eLearning系统上传，**按时上传就有分**）
- **期末考试（70%）**

教材

- 《算法导论》(第3版)(美)科尔曼等著, 殷建平等译, 机械工业出版社



课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP, NP 完全性，近似算法

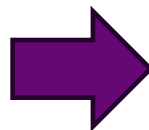
解递归式的几种方法

- 递归树
- 代入法
 - 直接代入
 - 换元代入
- 假设验证法
- 主定理

代入法

$$\begin{cases} T(n) = T(n-1) + n - 1 \\ T(1) = 0 \end{cases}$$

直接代入

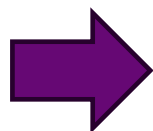


$$\begin{aligned} T(n) &= T(n-1) + n - 1 \\ &= (T(n-2) + n - 2) + n - 1 \\ &= (T(n-3) + n - 3) + n - 2 + n - 1 = \dots \\ &= T(1) + 1 + 2 + 3 \dots + n - 1 \\ &= n(n-1)/2 \end{aligned}$$

代入法

$$\begin{cases} T(n) = 2T(n/2) + n - 1 \\ T(1) = 0 \end{cases}$$

换元代入
 $n = 2^k$



$$T(n) = 2T(2^{k-1}) + 2^k - 1$$

$$= 2[2T(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1$$

$$= 2^2T(2^{k-2}) + 2^k - 2 + 2^k - 1$$

$$= 2^3[T(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1$$

$$= \dots$$

$$= 2^kT(1) + k2^k - (2^{k-1} + 2^{k-2} + 2^{k-3} + \dots + 2 + 1)$$

$$= k2^k - 2^{k+1}$$

$$= n \lg n - n + 1$$

假设验证法

- $T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + \Theta(n)$
- 先猜，后代入验证，设 $\Theta(n) = an$
- (1) 设 $T(n) = C$ (验证不通过)
 - $T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + \Theta(n) = \frac{2}{n} C(n-1) + an = 2C - 2C/n + an$
 - $2C - 2C/n + an = C?$
- (2) 设 $T(n) = cn$ (验证不通过)
 - 右边 $= \frac{2}{n} \sum_{i=1}^{n-1} ci + \Theta(n) = \frac{2c}{n} \frac{n(n-1)}{2} + an = cn - c + an$
 - $cn - c + an = cn \Rightarrow an - c = 0?$

假设验证法

- $T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + \Theta(n)$
- 先猜，后代入验证，设 $\Theta(n) = an$
- (3) 设 $T(n) = cn^2$ (验证不通过)
 - 右边 $= \frac{2}{n} \sum_{i=1}^{n-1} ci^2 + \Theta(n) = \frac{2c}{n} \frac{(n-1)n(2n-1)}{6} + an = \frac{2c}{3}n^2 - cn + \frac{c}{3} + an$
 - $\frac{2c}{3}n^2 - cn + \frac{c}{3} + an = cn^2 \Rightarrow \frac{c}{3}n^2 + cn - \frac{c}{3} - an = 0?$
- (4) 设 $T(n) = cn \log n$
 - 右边 $= \frac{2}{n} \sum_{i=1}^{n-1} ci \log i + \Theta(n) = \frac{2c}{n} \left[cn \log_2 n - \frac{c}{2 \ln 2} n + O(\log n) \right] + \Theta(n) = cn \log n - \frac{c}{2 \ln 2} n + \Theta(n) + O(\log n) = cn \log n + \Theta(n) = \Theta(n \log n)$

假设验证法：三看

- 看阶：代回后，主导增长阶是否一致
- 看系数：同阶时，最高阶项系数是否能自治
- 看低阶项：若只判断渐近阶，严格低阶项可忽略

求解一类递归式的通用方法：主定理

- 求解递归方程

$$T(n) = aT(n/b) + f(n)$$

- a 分解后的子问题个数
- $T(\frac{n}{b})$ 分解后的子问题规模
- $f(n)$ 分解和组合子问题解的开销

主定理

- **主定理：**

- 设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为递归式，且 $T(n) = aT(n/b) + f(n)$

- **则有以下结果：**

1. 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \lg n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对于某个常数 $c < 1$ 和充分大的 n 有 $af(n/b) \leq cf(n)$, 那么 $T(n) = \Theta(f(n))$

主定理的推导

$$T(n) = aT(n/b) + f(n)$$

$$\text{设 } n = b^k \Rightarrow k = \log_b n$$

$$= a[aT\left(\frac{n}{b}\right) + f\left(\frac{n}{b}\right)] + f(n) = a^2T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n)$$

$$= \dots$$

$$= a^kT\left(\frac{n}{b^k}\right) + a^{k-1}f\left(\frac{n}{b^{k-1}}\right) + \dots + af\left(\frac{n}{b}\right) + f(n)$$

$$= a^kT(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$\text{设 } T(1) = c$$

$$= cn^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$a^{\log_b n} = n^{\log_b a}$$

主定理1的推导

$$T(n) = cn^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$\varepsilon > 0$$

$$f(n) = O(n^{\log_b a - \varepsilon})$$

$$= cn^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right)$$

$$\frac{1}{(b^{\log_b a - \varepsilon})^j} = \frac{(b^\varepsilon)^j}{(b^{\log_b a})^j} = \frac{(b^\varepsilon)^j}{a^j}$$

$$= cn^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right)$$

$$= cn^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right)$$

$$= cn^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right)$$

$$b^{\varepsilon \log_b n} = b^{\log_b n^\varepsilon} = n^\varepsilon$$

$$T(n) = \Theta(n^{\log_b a})$$

$$= cn^{\log_b a} + O(n^{\log_b a - \varepsilon} n^\varepsilon) = O(n^{\log_b a}) \quad \text{同时: } T(n) \geq cn^{\log_b a} \Rightarrow T(n) = \Omega(n^{\log_b a}).$$

主定理2的推导

$$T(n) = cn^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$f(n) = \Theta(n^{\log_b a})$$

$$= cn^{\log_b a} + \Theta\left(\sum_{j=0}^{k-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right)$$

$$= cn^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a})^j}\right)$$

$$= cn^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{a^j}\right)$$

$$= cn^{\log_b a} + \Theta(n^{\log_b a} \lg n) = \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg n)$$

$$= \Theta(n^{\log_b a} \lg n)$$

主定理3的推导

$$T(n) = cn^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$\leq cn^{\log_b a} + \sum_{j=0}^{k-1} c^j f(n)$$

$$= cn^{\log_b a} + \sum_{j=0}^{\log_b n - 1} c^j f(n)$$

$$= cn^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1}$$

$$= cn^{\log_b a} + \Theta(f(n)) = \Theta(f(n))$$

$$c < 1$$

$$\varepsilon > 0$$

$$f(n) = \Omega(n^{\log_b a + \varepsilon})$$

$$af(n/b) \leq cf(n)$$

$$a^j f\left(\frac{n}{b^j}\right) \leq a^{j-1} cf\left(\frac{n}{b^{j-1}}\right) \leq ca^{j-1} f\left(\frac{n}{b^{j-1}}\right) \leq \dots \leq c^j f(n)$$

常数

现在看递归树第 j 层的总代价:

$$a^j f\left(\frac{n}{b^j}\right).$$

把条件中的 n 换成

$$\frac{n}{b^{j-1}},$$

得到:

$$af\left(\frac{n/b^{j-1}}{b}\right) \leq cf\left(\frac{n}{b^{j-1}}\right),$$

也就是:

$$af\left(\frac{n}{b^j}\right) \leq cf\left(\frac{n}{b^{j-1}}\right).$$

两边乘 a^{j-1} :

$$a^j f\left(\frac{n}{b^j}\right) \leq ca^{j-1} f\left(\frac{n}{b^{j-1}}\right)$$

然后再对右边继续用同一个条件:

$$a^{j-1} f\left(\frac{n}{b^{j-1}}\right) \leq ca^{j-2} f\left(\frac{n}{b^{j-2}}\right).$$

所以:

$$\begin{aligned} a^j f\left(\frac{n}{b^j}\right) &\leq ca^{j-1} f\left(\frac{n}{b^{j-1}}\right) \\ &\leq c^2 a^{j-2} f\left(\frac{n}{b^{j-2}}\right) \\ &\leq c^3 a^{j-3} f\left(\frac{n}{b^{j-3}}\right) \\ &\vdots \\ &\leq c^j f(n). \end{aligned}$$

所以最终得到:

$$a^j f\left(\frac{n}{b^j}\right) \leq c^j f(n)$$

主定理应用

- $T(n) = 4T(n/2) + n$

若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2, f(n) = n$$

$$\varepsilon = 1, f(n) = O(n^{\log_b a - \varepsilon}) = O(n)$$

$$T(n) = \Theta(n^2)$$

主定理应用

- $T(n) = 4T(n/2) + n^2$

若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \lg n)$

$$\begin{aligned} a = 4, b = 2 &\Rightarrow n^{\log_b a} = n^2, f(n) = n^2 \\ f(n) &= \Theta(n^{\log_b a}) = \Theta(n^2) \\ T(n) &= \Theta(n^2 \lg n) \end{aligned}$$

主定理应用

- $T(n) = 4T(n/2) + n^3$

若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对于某个常数 $c < 1$ 和充分大的 n 有 $af(n/b) \leq cf(n)$, 那么 $T(n) = \Theta(f(n))$

$$\begin{aligned} a = 4, b = 2 &\Rightarrow n^{\log_b a} = n^2, f(n) = n^3 \\ f(n) = n^3 &= \Omega(n^{\log_b a + \varepsilon}) \\ af\left(\frac{n}{b}\right) \leq cf(n) &\Rightarrow \frac{n^3}{2} \leq cn^3 \text{ 当 } c = \frac{1}{2} < 1 \\ T(n) &= \Theta(n^3) \end{aligned}$$

主定理应用

- $T(n) = 2T(n/2) + n \log n$
- 能否用主定理求解？

主定理应用

- $a = b = 2 \Rightarrow n^{\log_b a} = n, f(n) = n \log n$
- 不存在 $\varepsilon > 0$ 使得 $n \log n = \Omega(n^{1+\varepsilon})$, n^ε 增长速度远快于 $\log n$
- 不存在 $c < 1$ 使得 $2f\left(\frac{n}{2}\right) \leq cf(n) \Rightarrow n \log n - n \leq cn \log n \Rightarrow \log n \leq \frac{1}{1-c}$?

若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$

不满足

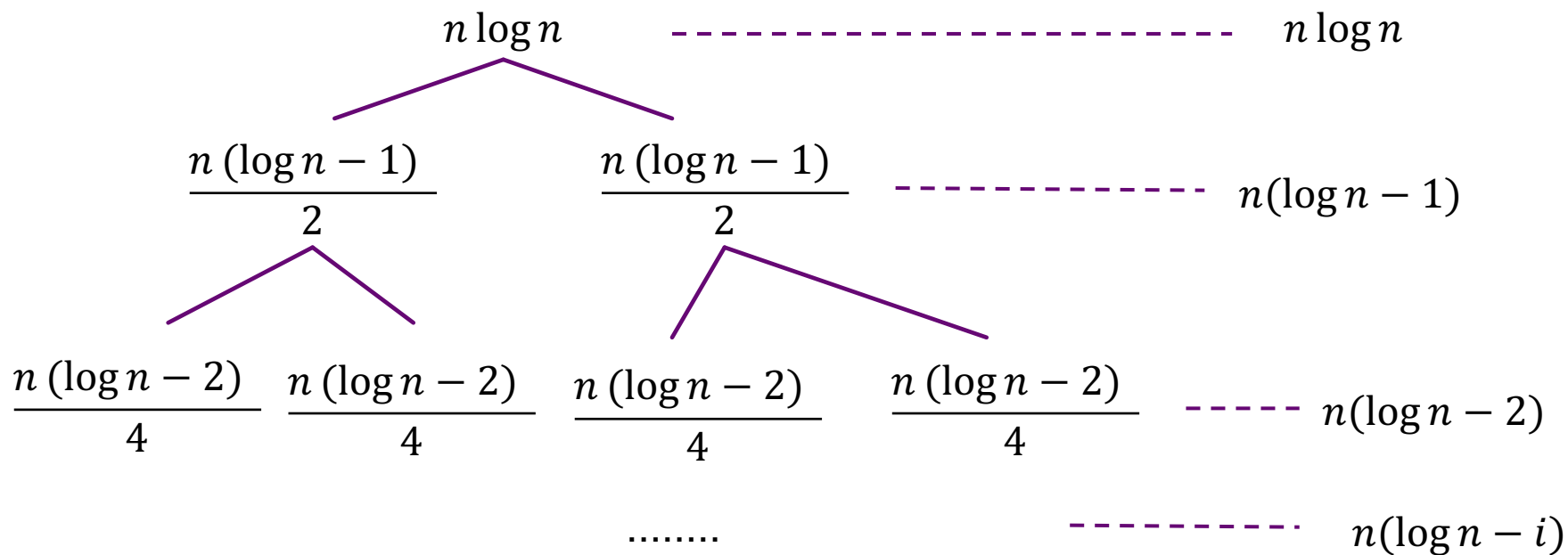
若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \lg n)$

不满足

若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对于某个常数 $c < 1$ 和充分大的 n 有 $af(n/b) \leq cf(n)$, 那么 $T(n) = \Theta(f(n))$

不满足

递归树



$$T(n) = n \sum_{i=0}^{\log n - 1} (\log n - i) + \Theta(n)$$

$$= n \log n \cdot \log n - n \sum_{i=0}^{\log n - 1} i + \Theta(n)$$

令 $L = \log_2 n$, 递归树各层是:

层数	该层总代价
0	$n \log n$
1	$n(\log n - 1)$
2	$n(\log n - 2)$
$\vdots$	$\vdots$
$L - 1$	n
L	$\Theta(n)$ (n 个叶子, 每个 $T(1) = \Theta(1)$)

二分检索算法

```
BINARY-SEARCH( $T, l, r, x$ )  
   $l \leftarrow 1; r \leftarrow n$   
  while  $l \leq r$  do  
     $m \leftarrow \lfloor (l + r) / 2 \rfloor$   
    if  $T[m] = x$  then  
      return  $m$   
    else if  $T[m] > x$  then  
       $r \leftarrow m - 1$   
    else  
       $l \leftarrow m + 1$   
  return  $0$ 
```

输入： 排好序的数组 T , 下标 l, r , 数 x

输出： 0 或者 m

例：二分检索

2	3	5	6	9	11	12
---	---	---	---	---	----	----

4

第一次比较：4 < 6

2	3	5	6	9	11	12
---	---	---	---	---	----	----

4

第二次比较：4 > 3

2	3	5	6	9	11	12
---	---	---	---	---	----	----

4

第三次比较：4 < 5

最坏情况下的时间复杂度

- 对排序好的数组，二分检索最坏情况下时间复杂度是 $\Theta(\log n)$
- 因为每比较一次，搜索范围都会缩小一半： $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$

平均情况下的时间复杂度：输入的可能性

- 假设数组长度 $n = 2^k - 1$

x 在数组 T 中

$$x = T[1]$$

$$x = T[2]$$

$$x = T[3]$$

$\vdots$

$$x = T[n]$$

n 种输入

x 不在数组 T 中

$$x < T[1]$$

$$T[1] < x < T[2]$$

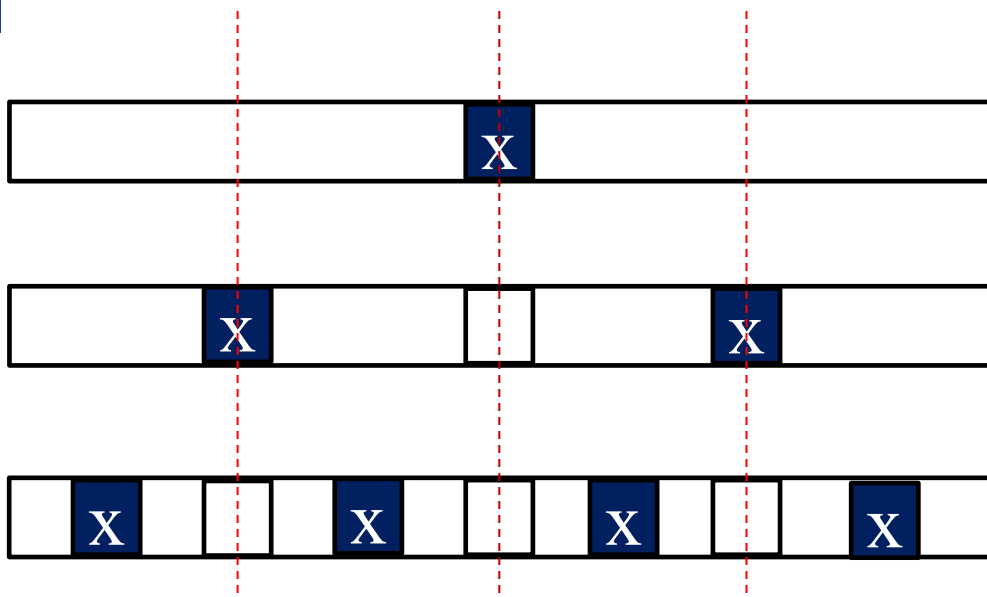
$\vdots$

$$T[n-1] < x < T[n]$$

$$x > T[n]$$

$n+1$ 种输入

平均情况下的时间复杂度：全部比较次数



假设数组长度 $n = 2^k - 1$

需要比较 k 次的输入有: $2^{k-1} + n + 1$ 种

- 2^{k-1} 种输入(在数组中)
- $n + 1$ 种输入(不在数组中)

$\Rightarrow 2^{t-1}$ 种输入 比较 t 次 ($t = 1, 2, \dots, k - 1$)

1种输入只比较1次

2个输入只比较2次

4个输入只比较3次

比较次数 查找成功的元素数

1

1

2

2

3

4

$\vdots$

$\vdots$

$k - 1$

2^{k-2}

k

2^{k-1}

平均情况下的时间复杂度：全部比较次数

第1次比较：先比较中间元素 4

1	2	3	4 X	5	6	7
---	---	---	--------	---	---	---

第2次比较：若未找到 4，则比较 2 或 6

1	2 X	3	4	5	6 X	7
---	--------	---	---	---	--------	---

第3次比较：再比较 1、3、5、7

1 X	2	3 X	4	5 X	6	7 X
--------	---	--------	---	--------	---	--------

查找失败共有 $n + 1 = 8$ 种区间：

$(-\infty, 1), (1, 2), (2, 3), (3, 4),$
 $(4, 5), (5, 6), (6, 7), (7, +\infty)$

比较 1 次： 1 个元素 (4)

比较 2 次： 2 个元素 (2, 6)

比较 3 次： 4 个元素 (1, 3, 5, 7)

查找失败： 8 种情况，也都比较 3 次

因此：

1 次比较： 1 种情况

2 次比较： 2 种情况

3 次比较： $4 + 8 = 12$ 种情况

所以这里只写 $t = 1, 2, \dots, k - 1$ ，
是因为 $t = k = 3$ 的情况被单独统计。

平均情况下的时间复杂度

- 假设 $n = 2^k - 1$ ，且 $2n + 1$ 种等价查找情形等概率

$$\begin{aligned} T(n) &= \frac{1}{2n+1} \left[\sum_{t=1}^{k-1} t2^{t-1} + k(2^{k-1} + n + 1) \right] \\ &= \frac{1}{2n+1} \left[\sum_{t=1}^k t2^{t-1} + k(n+1) \right] \\ &= \frac{1}{2n+1} [(k-1)2^k + 1 + k(n+1)] \\ &= \frac{k2^k - 2^k + 1 + k2^k}{2n+1} = \frac{k2^{k+1} - (2^k - 1)}{2^{k+1} - 1} \\ &\approx k - \frac{1}{2} = \lfloor \lg n \rfloor + \frac{1}{2} \end{aligned}$$

分治算法

- 分治算法(Divide and Conquer)
 - 将原始问题划分或者归结规模较小的子问题
 - 递归求解每个子问题
 - 将子问题的解合并得到原问题的解

注意：

1. 子问题与原始问题性质完全一样
2. 子问题之间可彼此独立地求解
3. 递归停止时子问题可直接求解

分治的精髓不是“把问题切开”，而是找到一种方式，让大问题在小规模上重复自己。

分治算法的复杂度分析

- 递归式是分析递归算法时间复杂度的通用工具；分治算法是其中非常典型的一类。
- 分治算法的两类主要递归式

$$T(n) = \sum_{i=1}^k a_i T(n - i) + f(n)$$

$$T(n) = aT(n/b) + f(n)$$

- 求解方法
 - 第一类方程：代入法、递归树、假设验证法
 - 第二类方程：代入法、递归树、主定理

见过的分治算法

- 归并排序的最坏情况时间复杂度

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & n > 1 \end{cases}$$

- 二分检索的最坏情况时间复杂度

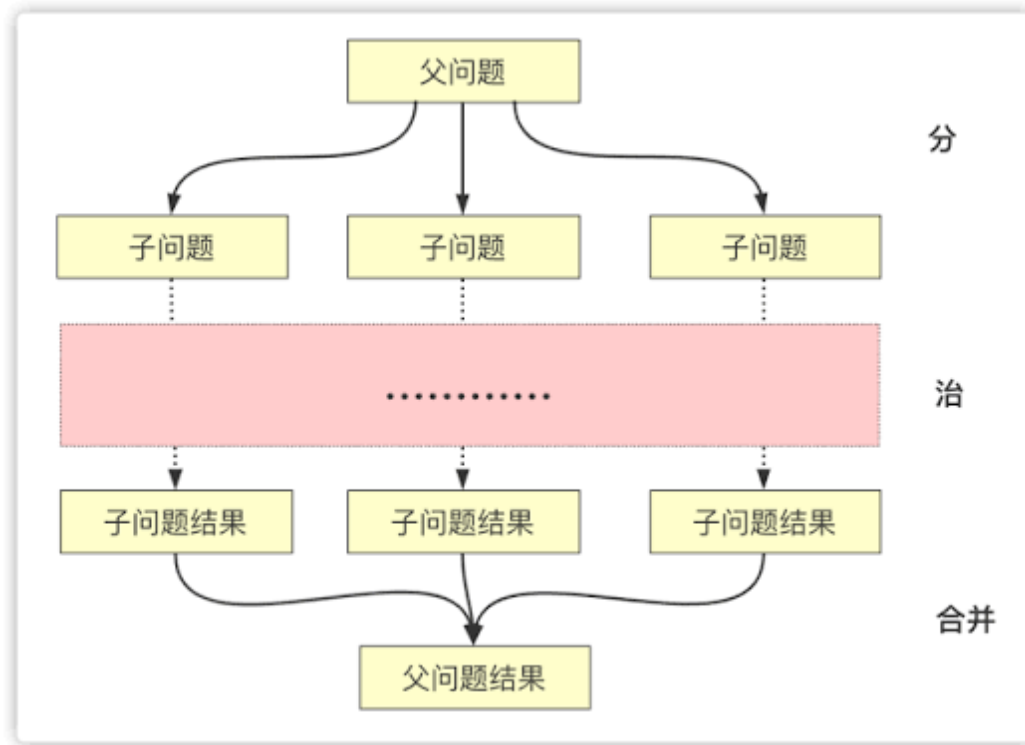
$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ T\left(\frac{n}{2}\right) + \Theta(1) & n > 1 \end{cases}$$

- 快速排序的最坏情况时间复杂度

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ T(n-1) + \Theta(n) & n > 1 \end{cases}$$

分治算法的应用

- 到目前为止，我们看到二分检索、归并排序、快速排序的递归式长得都不一样。但如果把公式拿掉，它们其实都在做同一件事：**避免一次性解决整个大问题，而是不断把问题缩小。**
- 分治思想广泛应用于搜索、排序、数值计算和并行计算等问题，并常常带来更高效的算法



矩阵乘法

- 输入: $A = [a_{ij}], B = [b_{ij}], i, j = 1, 2, 3, \dots, n$
- 输出: $C = [c_{ij}] = A \cdot B, i, j = 1, 2, 3, \dots, n$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & & c_{2n} \\ \vdots & & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & & b_{2n} \\ \vdots & & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

标准矩阵乘法

```
def matrix_multiply(A, B):  
    n = len(A)  
  
    # 初始化 n × n 的结果矩阵  
    C = [[0 for _ in range(n)] for _ in range(n)]  
  
    for i in range(n):  
        for j in range(n):  
            for k in range(n):  
                C[i][j] += A[i][k] * B[k][j]  
  
    return C
```

时间复杂度是 $\Theta(n^3)$

用分治思想优化矩阵乘法

- 将矩阵分块

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$r = ae + bg$$

$$s = af + bh$$

$$t = ce + dg$$

$$u = cf + dh$$

递归子问题

8 次 $(n/2) \times (n/2)$ 子矩阵的乘法
4 次 $(n/2) \times (n/2)$ 子矩阵的加法

算法的递归式

$$T(n) = 8T(n/2) + 4(n/2)^2 = 8T(n/2) + \Theta(n^2)$$

子问题个数：
8个子问题

子问题规模：
乘法规模减半

合并子问题的解：
矩阵加法复杂度

最坏情况下的时间复杂度是多少？

最坏情况下的时间复杂度

- 用主定理:

$$a = 8, b = 2, n^{\log_b a} = n^{\log_2 8} = n^3$$

- 而: $f(n) = \Theta(n^2) = O(n^{3-1})$

- 属于 Case 1, 所以:

- $T(n) = \Theta(n^3)$

若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$

能不能少做一次乘法？

$$T(n) = 8T(n/2) + \Theta(n^2)$$

加法只有 $\Theta(n^2)$ ，已经不是瓶颈。

如果想突破 n^3 ，该改哪里？

Strassen 算法的核心：用加法换乘法

- 用 7 次递归乘法来实现了 2×2 子矩阵相乘。

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

7 次乘法，18 次加法/减法

Strassen 怎么做到的?

- 普通方法想分别算: $ae, bg, af, bh, ce, dg, cf, dh$
- 但 Strassen 的想法是: 我们不需要知道每个乘积, 只需要最后得到 r, s, t, u 。
- 例如: $r = ae + bg$, Strassen 构造一些“混合乘法”: $M_1 = (a + d)(e + h)$
- 一次乘法里同时包含: ae, ah, de, dh
- 再用其他 M_i 加减, 把多余项消掉, 最后留下:

$$r = ae + bg$$

- 核心思想: 用更多加减法, 换掉一次递归乘法

例如取:

$$M_1 = (a + d)(e + h)$$

$$M_4 = d(g - e)$$

$$M_5 = (a + b)h$$

$$M_7 = (b - d)(g + h)$$

然后:

$$r = M_1 + M_4 - M_5 + M_7.$$

展开看看:

$$M_1 = ae + ah + de + dh$$

$$M_4 = dg - de$$

$$-M_5 = -ah - bh$$

$$M_7 = bg + bh - dg - dh$$

全部加起来, 很多项消掉:

$$ah, de, dh, dg, bh,$$

最后只剩:

$$ae + bg = r.$$

Strassen算法的时间复杂度

- $T(n) = 7T(n/2) + 18n^2$
- $a = 7, b = 2, n^{\log_b a} = n^{\log_2 7} = n^{2.81}$
- $T(n) = \Theta(n^{2.81})$

若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$

矩阵乘法：从算法问题到 **AI** 核心计算

- $C = AB, C_{ij} = \sum_k A_{ik} B_{kj}$
- 前面我们已经看到：
 - 朴素矩阵乘法： $\Theta(n^3)$
 - Strassen： $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$
- 但矩阵乘法不只是算法课里的经典问题：
- 现代 AI 的大量计算，本质上都是矩阵运算

AI 模型里，矩阵乘法无处不在

全连接 / Linear Layer

$$Y = XW + b$$

其中：

X : 输入, W : 模型参数

这一层的核心计算就是：

$$XW$$

Transformer: Q、K、V 投影

$$Q = XW_Q, K = XW_K, V = XW_V$$

Transformer 层有多次矩阵乘法。

Attention 计算：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

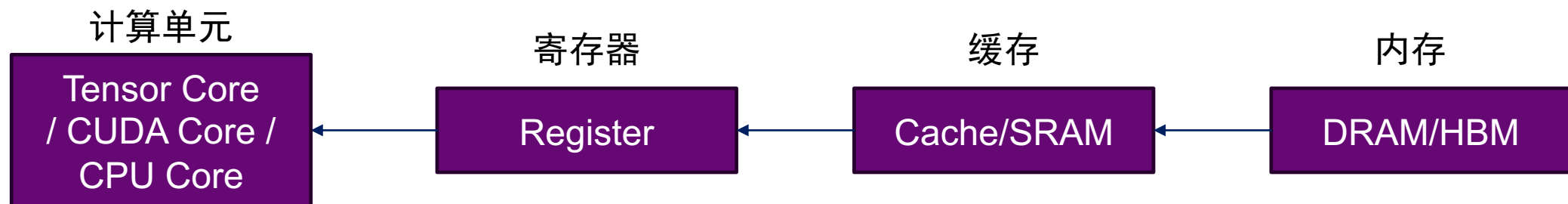
里面有： QK^T 和 $\text{softmax}(\cdot)V$ 两个核心矩阵乘法

矩阵乘法的效率，直接影响神经网络训练和推理的效率

降低复杂度，是唯一的优化方式吗？

- 普通矩阵乘法： $\Theta(n^3)$, Strassen: $\Theta(n^{2.807})$
- 如果我们仍然做 $\Theta(n^3)$ 次运算，能不能跑得更快？

Strassen 优化的是计算次数；现代高性能矩阵计算还要优化数据搬运。



越靠近计算单元，越快但越小

越远离计算单元，越大但越慢

性能差异的来源

```
def matrix_multiply(A, B):
    n = len(A)

    # 初始化  $n \times n$  的结果矩阵
    C = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]

    return C
```

渐近复杂度刻画计算量
但没有刻画数据移动和硬件利用率

- **BLAS (Basic Linear Algebra Subprograms):**
 - 基础线性代数运算的标准规范（API接口），它本身只是一个标准，由 Netlib 提供了官方的参考实现（通常用 Fortran 编写，速度一般）
- **MKL (Intel OneAPI Math Kernel Library):**
 - 是 Intel 推出的针对自家 **CPU**（x86架构）高度优化的商业高性能数学库。它完整实现了 BLAS、LAPACK 等规范，内部针对 Intel 处理器指令集（如 AVX-512、AMX）进行了极致的汇编与并行优化
- **cuBLAS (CUDA Basic Linear Algebra Subroutines):**
 - 是 NVIDIA 推出的针对自家 **GPU** 高度优化的官方 **BLAS** 库。它实现了标准的 BLAS 接口，但运行在 NVIDIA GPU 上，能够利用 CUDA 核心与 Tensor Core 提供极致的并行算力和加速

分块矩阵乘法 (**Block Matrix Multiplication**)

- 把矩阵分块:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- 计算: $C_{ij} = \sum_k A_{ik} B_{kj}$
- 这里的 A_{ik}, B_{kj}, C_{ij} 小矩阵块
- **重点: Blocking 不是为了减少乘法次数, 而是为了提高数据复用,** 加载一次 → 计算多次
- **Temporal Locality:** 时间局部性, 刚刚访问过的数据, 很快再次访问。
- **Spatial Locality:** 空间局部性, 访问一个位置以后, 很可能访问附近位置。

基于图块的计算 (**Tile-based Computation**)

- 层次:

Matrix → Block → Tile → Register / SIMD / Tensor Core

- Tile 的目的: 让每个小计算单元处理合适大小的数据
- 算法里问“怎么拆问题”，系统里还要问“拆成多大最适合硬件”

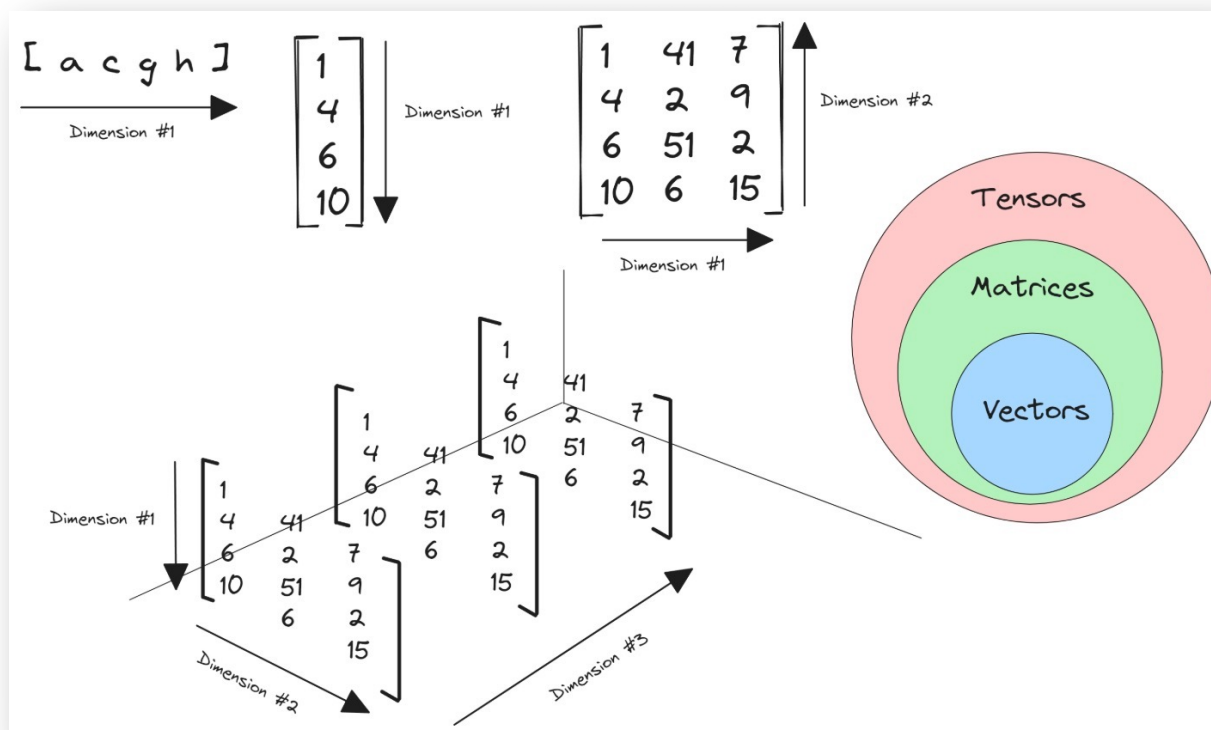
GEMM：现代数值计算的核心算子

- General Matrix Multiply: $C = \alpha AB + \beta C$
- Blocking: 提高 Cache 复用
- Tiling: 适配计算单元
- Vectorization: 一次处理多个数据
- Parallelism: 多个核心同时计算

GEMM= Blocking + Tiling + Vectorization + Parallelism

从矩阵到Tensor

- 矩阵: $A[i, j]$
- Tensor: $X[b, h, s, d]$
 - b : batch
 - h : head
 - s : sequence
 - d : hidden dimension
- Tensor 一样可以: 分块 / 切片 / Tile



分块思想，仍在改变 **AI** 性能

- 传统 attention: $QK^T \rightarrow$ 写回大矩阵 $\rightarrow \text{Softmax} \rightarrow V$
- 中间产生大量数据搬运。
- FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022
 - 按 Tile / Block 分块计算 Attention
 - 尽量让中间结果驻留在片上高速存储 (SRAM)
 - 减少 缓存和内存 之间的数据读写
- 重点：不是通过降低理论 FLOPs 来加速，通过重新组织计算，降低 IO

卷积和多项式乘法

- $a = (a_0, a_1, a_2, \dots, a_{n-1}), b = (b_0, b_1, b_2, \dots, b_{m-1})$

向量和
($m = n$)

$$a + b = (a_0 + b_0, a_1 + b_1, a_2 + b_2, \dots, a_{n-1} + b_{n-1})$$

内积
($m = n$)

$$a \cdot b = (a_0 b_0, a_1 b_1, a_2 b_2, \dots, a_{n-1} b_{n-1})$$

卷积

$$a * b = (c_1, c_2, c_3, \dots, c_k)$$

$$c_k = \sum_{\substack{i+j=k \\ i \in [0, n-1] \\ j \in [0, m-1]}} a_i b_j, k = 0, 1, 2, \dots, m + n - 2$$

多项式乘法

- 多项式乘法 $C(x) = A(x)B(x)$

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{m-1}x^{m-1}$$

$$C(x) = a_0b_0 + (a_0b_1 + a_1b_0)x + \cdots + a_{n-1}b_{m-1}x^{m+n-2}$$

- 其中系数（卷积等价于多项式乘法计算）：

$$c_k = \sum_{\substack{i+j=k \\ i \in [0, n-1] \\ j \in [0, m-1]}} a_i b_j, k = 0, 1, 2, \dots, m+n-2$$

暴力计算： $O(n^2), m = n$

用分治思想优化多项式乘法

- 分解 ($n \times n$ 的卷积)

- $$A(x) = (a_0 + \dots + a_{\frac{n}{2}-1} x^{\frac{n}{2}-1}) + x^{\frac{n}{2}} (a_{\frac{n}{2}} + \dots + a_{n-1} x^{\frac{n}{2}-1}) = A_0 + x^{\frac{n}{2}} A_1$$

- $$B(x) = (a_0 + \dots + a_{\frac{n}{2}-1} x^{\frac{n}{2}-1}) + x^{\frac{n}{2}} (a_{\frac{n}{2}} + \dots + a_{n-1} x^{\frac{n}{2}-1}) = B_0 + x^{\frac{n}{2}} B_1$$

- 解决: $A_0 * B_0, A_0 * B_1, A_1 * B_0, A_1 * B_1$

- 合并:
$$A(x)B(x) = A_1 B_1 x^n + x^{\frac{n}{2}} (A_0 B_1 + A_1 B_0) + A_0 B_0$$

4 次 $n/2$ 次多项式乘法（递归），3 次 n 次多项式加法

递归式

$$T(n) = 4T(n/2) + O(n)$$

子问题个数：
4个子问题

子问题规模：
乘法规模减半

合并子问题的解：
矩阵加法复杂度

时间复杂度

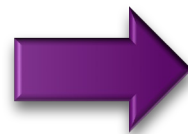
- $a = 4, b = 2, n^{\log_b a} = n^{\log_2 4} = n^2$
- 主定理1: 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
- $T(n) = \Theta(n^2)$
- 时间复杂度和暴力计算一样
- $A(x)B(x) = A_1B_1x^n + x^{\frac{n}{2}}(A_0B_1 + A_1B_0) + A_0B_0$
- 需要优化 分治后矩阵乘法 的次数

Karatsuba算法

$$A(x)B(x) = A_1B_1x^n + x^{\frac{n}{2}} \boxed{(A_0B_1 + A_1B_0)} + A_0B_0$$

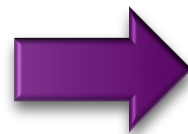
$$A_0B_1 + A_1B_0 = (A_0 + A_1)(B_0 + B_1) - A_1B_1 - A_0B_0$$

2次乘法, 1次加法



1次乘法, 4次加/减法

$$T(n) = 4T(n/2) + O(n)$$

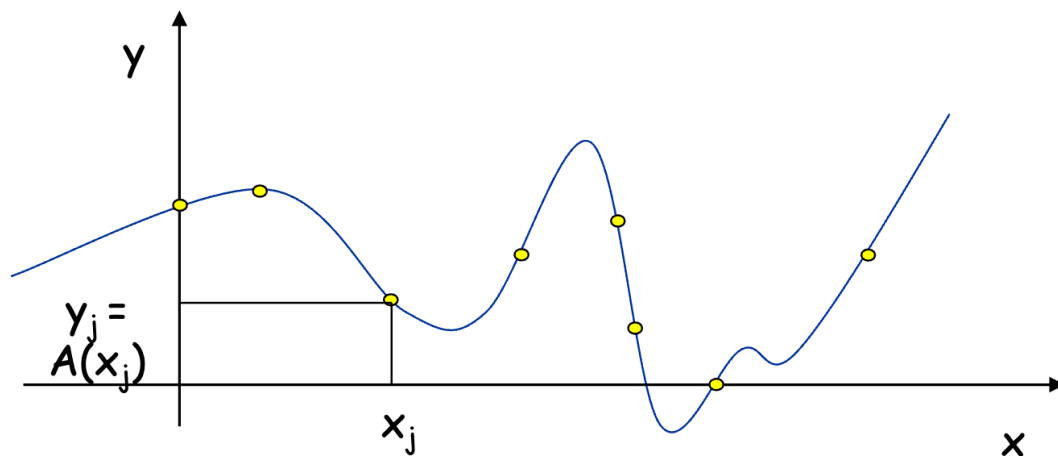


$$T(n) = 3T(n/2) + O(n)$$

$$T(n) = \Theta(n^{1.584...})$$

多项式的另一种表示：点值

- n 次多项式可以通过其在 $n + 1$ 个不同点上的值唯一确定（并且可以通过这些值进行插值）
- 如2个点确定一条直线



推论：与列出 $n - 1$ 次多项式 $A(x)$ 的相对的一种表示方法是给出该多项式在任意 n 个不同点上的值 $(x_i, A(x_i))$ ，其中 $i = 1, 2, \dots, n$

依赖点值表示：实现多项式加/乘法

$$A(x): (x_0, y_0), (x_1, y_1), (x_2, y_2) \dots (x_{n-1}, y_{n-1})$$

$$B(x): (x_0, y'_0), (x_1, y'_1), (x_2, y'_2) \dots (x_{n-1}, y'_{n-1})$$

多项式加法：时间复杂度 $O(n)$

$$C(x) = A(x) + B(x): (x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \dots (x_{n-1}, y_{n-1} + y'_{n-1})$$

多项式乘法 $C(x) = A(x) * B(x)$: **$2n - 2$ 次多项式，至少需要 $2n - 1$ 个点**

$$A(x): (x_0, y_0), (x_1, y_1), (x_2, y_2) \dots (x_{2n-2}, y_{2n-2})$$

$$B(x): (x_0, y'_0), (x_1, y'_1), (x_2, y'_2) \dots (x_{2n-2}, y'_{2n-2})$$

$$C(x) = A(x) * B(x): (x_0, y_0 y'_0), (x_1, y_1 y'_1), \dots (x_{n-1}, y_{2n-2} y'_{2n-2})$$

$2n-1$ 次乘法 ==> 时间复杂度 $O(n)$!

从点值出发：计算多项式系数

- 拉格朗日插值法： $(x_0, y_0), \dots (x_{n-1}, y_{n-1})$

$$P(x) = \sum_{i=0}^{n-1} L_i(x) \cdot y_i, \text{ 其中 } L_i(x) = \prod_{\substack{0 \leq j \leq n-1 \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

n次运算 n次运算

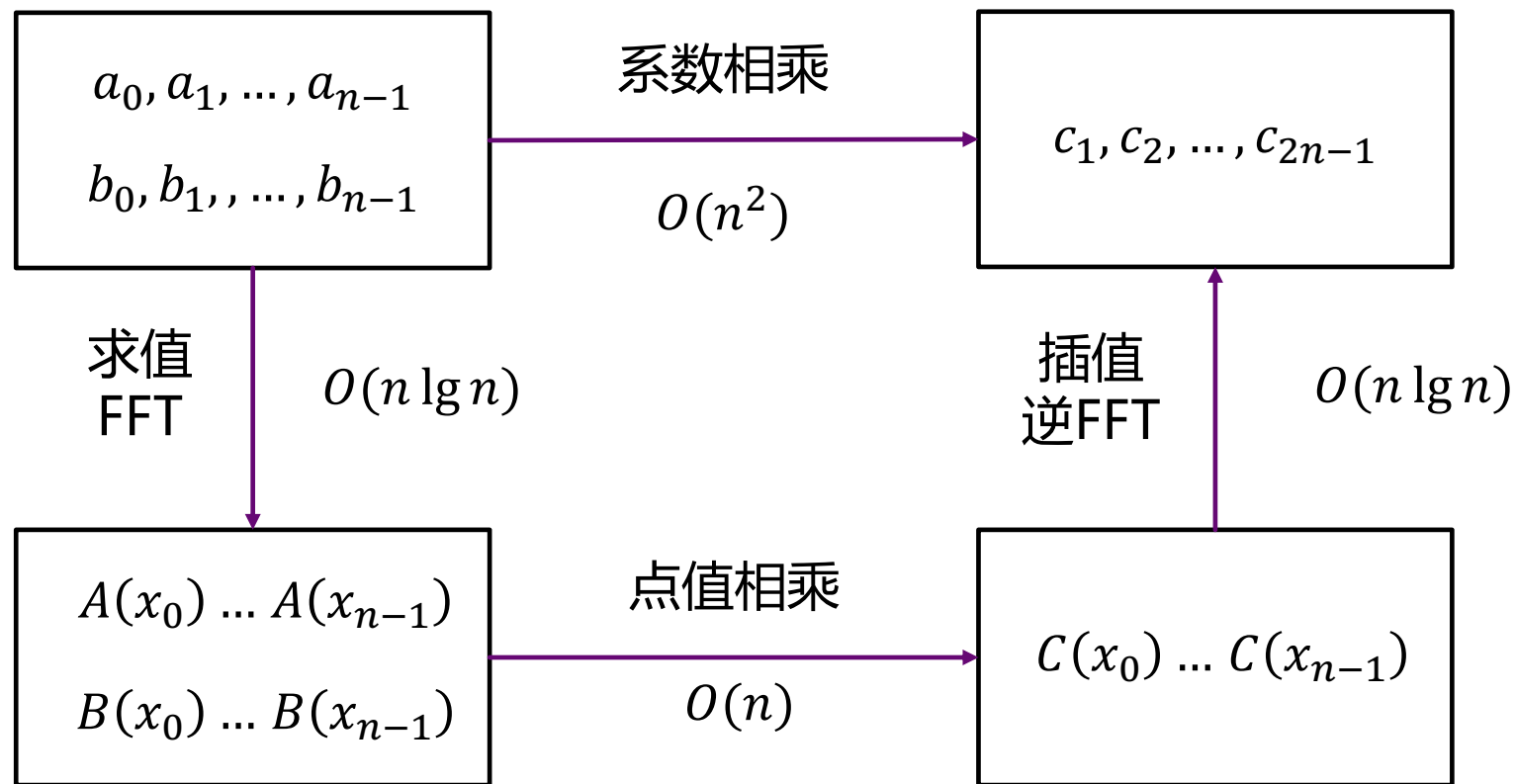
时间复杂度 $O(n^2)$

表示	相乘	求值/插值
系数	$O(n^2)$	$O(n)$
点值	$O(n)$	$O(n^2)$

拉格朗日插值法怎么得到的？

- 第一步，先构造一个“只认第 i 个点”的多项式： $L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ ，满足： $L_i(x_j) = \begin{cases} 1, & j = i \\ 0, & j \neq i \end{cases}$
 - 也就是说， L_i 在第 i 个点取 1，在其他所有已知点取 0。
- 第二步，用 y_i 给这些基函数加权： $P(x) = \sum_{i=0}^n y_i L_i(x)$ ，代入 $x = x_k$ ： $P(x_k) = \sum_i y_i L_i(x_k) = y_k$ 。
- 所以最终： $P(x) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$
- 先造一组“one-hot 多项式”：在自己的点上为 1、其他点上为 0；再按 y_i 加权相加。

快速傅立叶变换



为了优化求值和差值，需要快速傅里叶变换

算法思想

- 输入多项式
- $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$
- $B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{n-1}x^{n-1}$

对多项式 $A(x)$ 和 $B(x)$ 在 $2n - 1$ 个点处求值，转换成点值表示



相乘 $2n - 1$ 个点值，计算 $C(x_i) = A(x_i)B(x_i)$

$O(n)$



通过插值方法将 $(x, C(x))$ 在 $2n - 1$ 个点上的表示转换为 $C(x)$ 的系数表示

分治思想

- 分解(设 n 是偶数): 把 $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$ 拆分成 $A_{odd}(x)$ 和 $A_{even}(x)$
- $A_{odd}(x) = a_1 + a_3x + \cdots + a_{n-1}x^{n/2-1}$, $A_{even}(x) = a_0 + a_2x + \cdots + a_{n-2}x^{n/2-1}$
- 满足: $A(x) = A_{even}(x^2) + xA_{odd}(x^2)$
- 解决: 递归地求 $A_{even}(x_j^2)$ 和 $A_{odd}(x_j^2)$, $j = 0, 1, \dots, n-1$
- 合并: $A(x) = A_{even}(x_j^2) + x_jA_{odd}(x_j^2)$, $j = 0, 1, \dots, n-1$

每个点: $T(n) = 2T(n/2) + O(1) = O(n)$

n 个点: $n \cdot O(n) = O(n^2)$

举例

多项式有 8 项: $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_7x^7$.

第一次拆: $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$,

其中

$$A_{\text{even}}(z) = a_0 + a_2z + a_4z^2 + a_6z^3,$$

$$A_{\text{odd}}(z) = a_1 + a_3z + a_5z^2 + a_7z^3.$$

变量 $z = x^2$:

$A_{\text{even}}(z)$ 就是一个正常的 4 项多项式:

$$a_0 + a_2z + a_4z^2 + a_6z^3.$$

再拆: $A_{\text{even}}(z) = E_{\text{even}}(z^2) + zE_{\text{odd}}(z^2)$,

其中

$$E_{\text{even}}(w) = a_0 + a_4w,$$

$$E_{\text{odd}}(w) = a_2 + a_6w.$$

同理: $A_{\text{odd}}(z) = O_{\text{even}}(z^2) + zO_{\text{odd}}(z^2)$,

其中

$$O_{\text{even}}(w) = a_1 + a_5w,$$

$$O_{\text{odd}}(w) = a_3 + a_7w.$$

所以系数实际上是在不断这样拆:

$$[a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7]$$

第一次:

$$[a_0, a_2, a_4, a_6] [a_1, a_3, a_5, a_7]$$

第二次:

$$[a_0, a_4], [a_2, a_6], [a_1, a_5], [a_3, a_7].$$

再往下:

$$[a_0], [a_4], [a_2], [a_6], [a_1], [a_5], [a_3], [a_7].$$

减少递归调用时的计算开销

- 思想：选择 n 个点 $\pm x_0, \pm x_1, \dots, \pm x_{n/2-1}$
- $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$
- $A(-x) = A_{\text{even}}((-x)^2) - xA_{\text{odd}}((-x)^2)$
- 因此，可以通过一次调用的代价，获得 $\pm x_i$ **两个点上函数的值**
- 将 $n - 1$ 次多项式在 n 个点上评估的复杂度降为在 $n/2$ 个点上评估两个 $n/2 - 1$ 次多项式

举例

- 设 $A(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ ，不是只求一个点，而是一次要求 4 个单位根上的值：
 $A(1), A(i), A(-1), A(-i)$
- 拆成： $A(x) = E(x^2) + xO(x^2)$
- 其中 $E(z) = a_0 + a_2z, O(z) = a_1 + a_3z$
- 这 4 个点平方以后： $1^2 = 1, i^2 = -1, (-1)^2 = 1, (-i)^2 = -1$ ，原来 4 个评价点，对子多项式来说只剩： $1, -1$
- 解 $E(z) = a_0 + a_2z, O(z) = a_1 + a_3z$ ，得到 $E(1), E(-1), O(1), O(-1)$
- $A(1) = E(1) + O(1), A(-1) = E(1) - O(1), A(i) = E(-1) + iO(-1), A(-i) = E(-1) - iO(-1)$

$$4 \text{ 个点} \rightarrow \begin{cases} E: 2 \text{ 个点} \\ O: 2 \text{ 个点} \end{cases}$$



$$T(4) = 2T(2) + O(4)$$

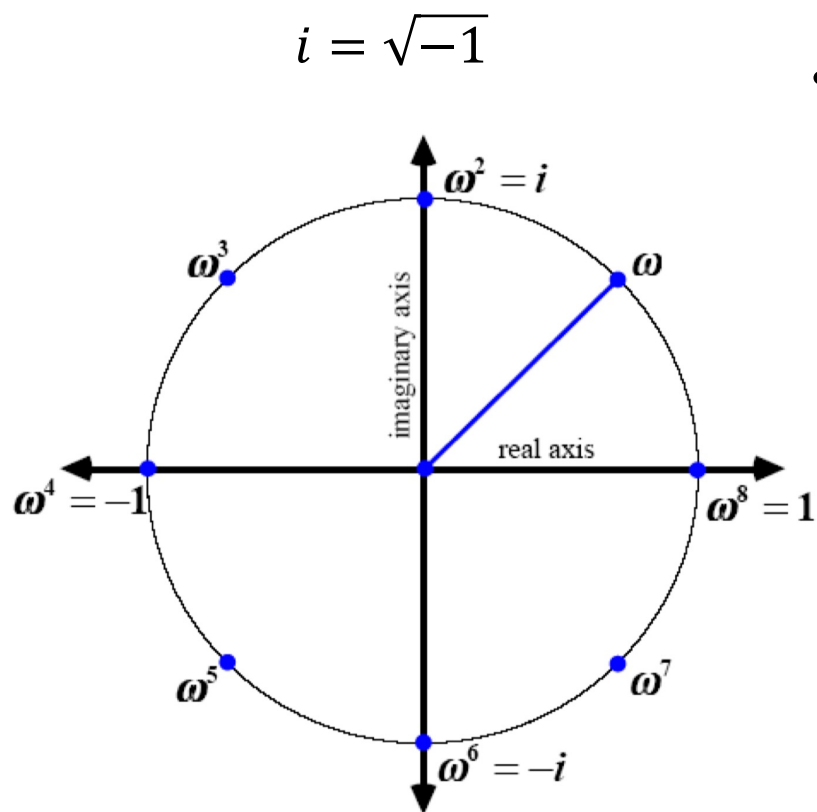


推广 $n = 8, 16, 32, \dots$

$$T(n) = 2T(n/2) + O(n)$$

需要选择 $\pm x_0, \pm x_1 \dots \pm x_{n/2-1}$ ，使得
 $x_0^2, x_1^2, \dots, x_{n/2-1}^2$ 是 $n/4$ 个正负对

单位复数根



$\omega_8 = e^{2\pi i/8}$
是一个 8 次单位复数根

- $\omega_n = e^{2\pi i/n}$ 是一个 n 次单位复数根 ($z^n = 1$ 的解, $z = e^{i\theta} = \cos \theta + i \sin \theta$), 满足 $(\omega_n)^n = 1, (\omega_n)^j \neq 1 (0 < j < n)$

折半定理: 如果 $n > 0$ 是偶数, 那么 n 个 n 次单位复数根的平方的集合就是 $n/2$ 个 $n/2$ 次单位复数根的集合

$$(\omega_n^j)^2 = e^{2 \cdot 2\pi i j / n} = e^{\frac{2\pi i j}{n/2}} = \omega_{n/2}^j$$

$$\{(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2\} = \{\omega_{n/2}^0, \dots, \omega_{n/2}^{n/2-1}\}$$

此外, 当 n 为偶数时, n 次单位复数根是正负对的:

$$\omega_n^{n/2} = -1 \Rightarrow \omega_n^{j+\frac{n}{2}} = -\omega_n^j (j = 0, \dots, n/2)$$

举例

具体写成：

$$\left\{1, \frac{1+i}{\sqrt{2}}, i, \frac{-1+i}{\sqrt{2}}, -, 1, \frac{-1-i}{\sqrt{2}}, -i \frac{1-i}{\sqrt{2}}\right\}$$

$\downarrow x^2$

$$\{1, i, -, 1-i\}$$

$\downarrow x^2$

$$\{1, -1\}$$

$\downarrow x^2$

$$\{1\}$$

这就是 FFT 能不断递归的关键：

$$\omega_n^j \xrightarrow{x^2} \omega_{n/2}^j$$

每平方一次，不同评价点的数量减半，而且剩下的仍然是一组单位根。

快速傅立叶变换

■ $FFT(A, n)$

1. 如果 $n = 1$, 返回 $A(1)$ 。
2. 将 A 分解为 A_{odd} 和 A_{even} 。
3. 调用 $FFT(A_{odd}, n/2)$ 和 $FFT(A_{even}, n/2)$ $2T(n/2)$
4. 计算:

$$\begin{aligned} A(\omega_n^j) &= A_{even}((\omega_n^j)^2) + \omega_n^j A_{odd}((\omega_n^j)^2) \\ A(\omega_n^{j+n/2}) &= A_{even}((\omega_n^j)^2) - \omega_n^j A_{odd}((\omega_n^j)^2) \end{aligned} \quad j = 0, \dots, n/2$$

$O(n)$

5. Return $A(\omega_n^0), A(\omega_n^1), \dots, A(\omega_n^{n-1})$

递归式: $T(n) = 2T\left(\frac{n}{2}\right) + O(n) \Rightarrow O(n \lg n)$

逆离散傅里叶变换

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega_n^{-jk}$$

原多项式:

$$A(x) = 1 + 2x + 3x^2 + 4x^3$$

所以原系数:

$$[a_0, a_1, a_2, a_3] = [1, 2, 3, 4].$$

前面 FFT 得到在四次单位根

$$1, i, -1, -i$$

上的点值:

$$[y_0, y_1, y_2, y_3] = [10, -, 2, -2i - 2 - 2 + 2i]$$

用公式计算系数。

四次单位根:

$$\omega_4 = i.$$

求 a_0
令 $k = 0$:

$$a_0 = \frac{1}{4} \sum_{j=0}^3 y_j \omega_4^{-j \cdot 0}.$$

由于:

$$\omega_4^0 = 1,$$

所以:

$$\begin{aligned} a_0 &= \frac{1}{4} [10 + (-2 - 2i) + (-2) + (-2 + 2i)] \\ &= \frac{1}{4} (4) \\ &= \boxed{1}. \end{aligned}$$

a_1, a_2, a_3 类似

向量卷积算法

1. 求 $A(\omega_n^0) \dots A(\omega_n^{n-1})$ 和 $B(\omega_n^0) \dots B(\omega_n^{n-1})$
2. 求 $C(\omega_n^j) = A(\omega_n^j)B(\omega_n^j), j = 0, 1, \dots, n-1$
3. 插值求 $C(x)$

时间复杂度分析：

第一步： $O(n \lg n)$
第二步： $O(n)$
第三步： $O(n \lg n)$

$\left. \begin{array}{l} \text{第一步： } O(n \lg n) \\ \text{第二步： } O(n) \\ \text{第三步： } O(n \lg n) \end{array} \right\} O(n \lg n)$

最小点对问题

输入

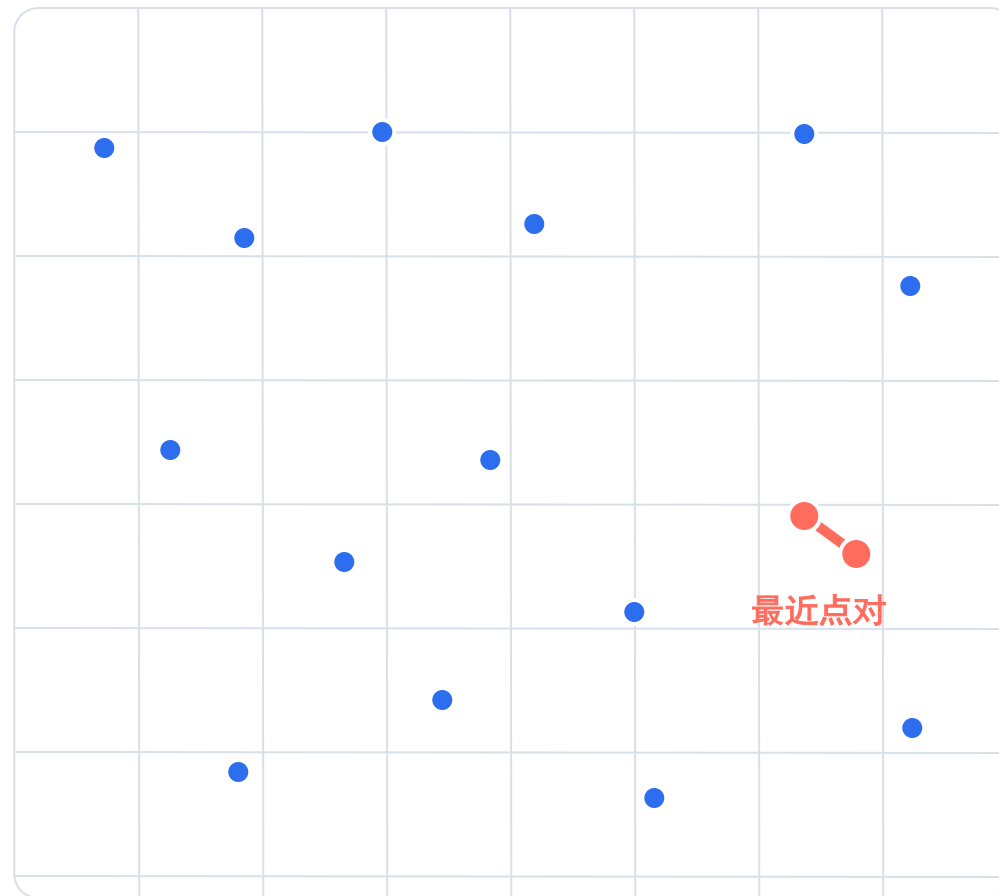
点集 $P = \{p_1, p_2, \dots, p_n\}$

目标

找到使 $d(p_i, p_j)$ 最小的点对

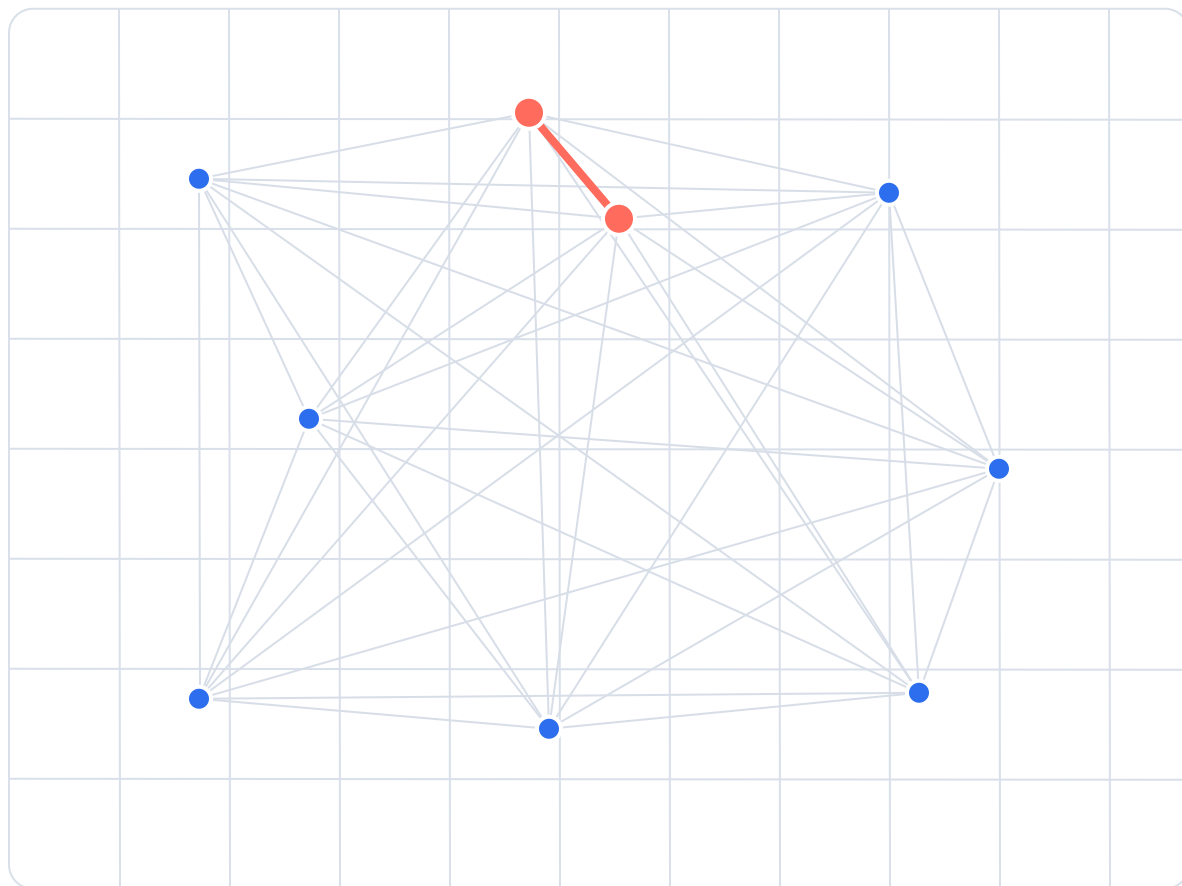
欧氏距离

$$d(p_i, p_j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]}$$



输出不仅是最小距离，也可以包含对应的两个点

暴力枚举的代价



比较次数

$$C(n, 2) = n(n - 1)/2$$

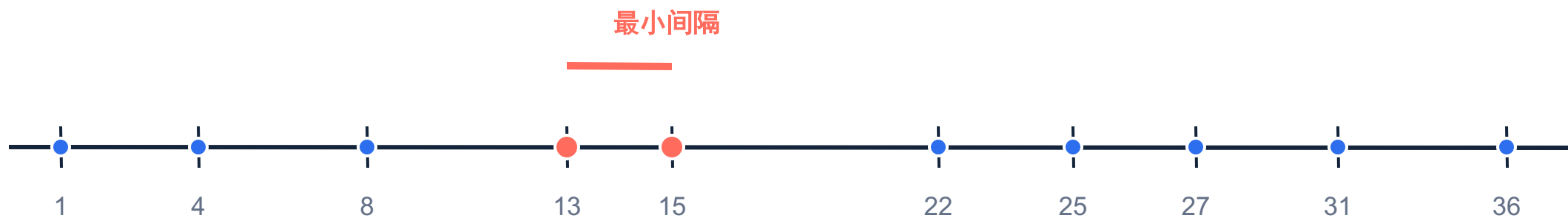
当 $n = 10,000$

49,995,000 次比较

时间复杂度 $\Theta(n^2)$

原因：没有利用点在平面上的顺序与局部性

一维直觉：排序让候选变少



排序

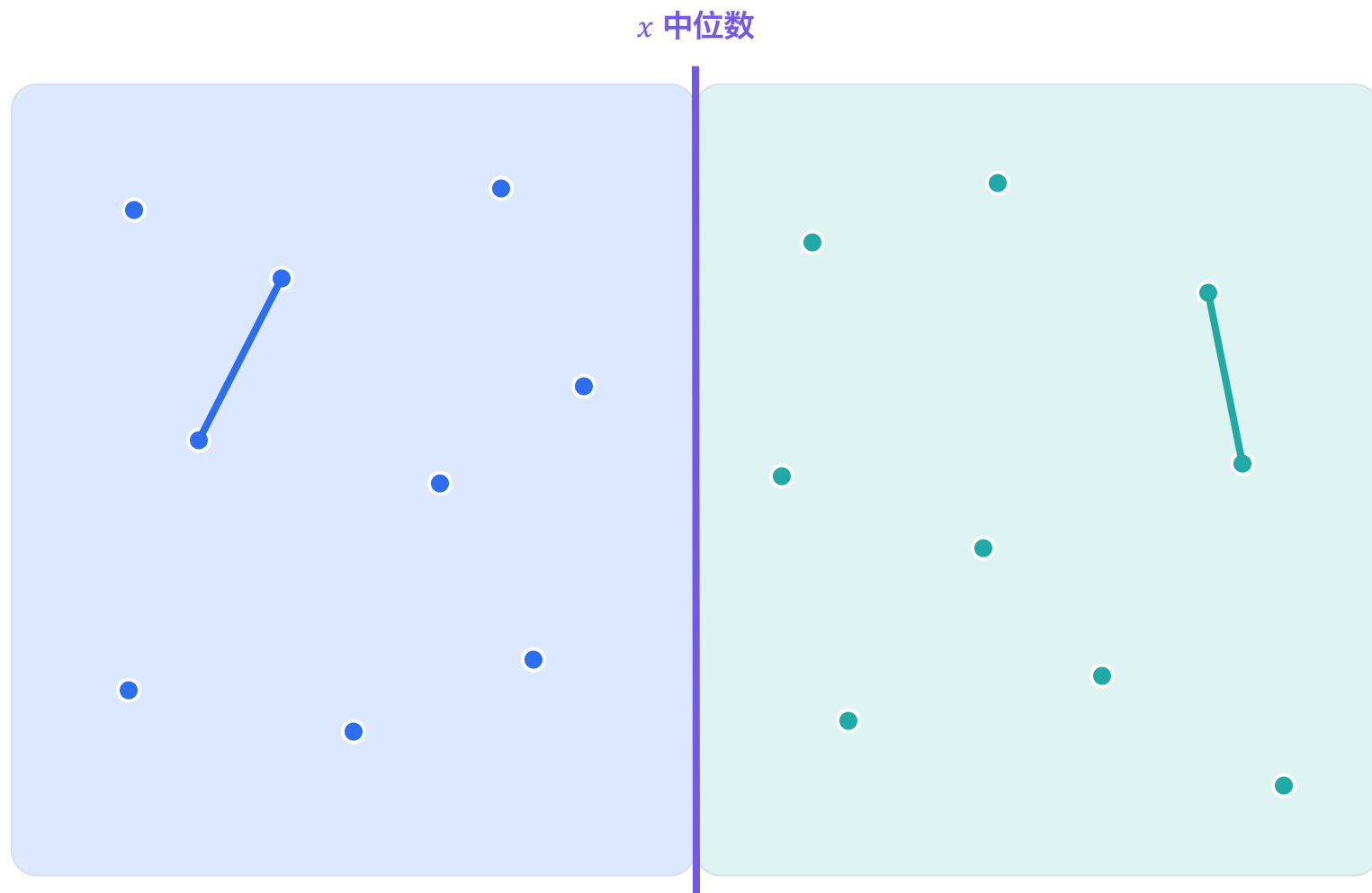
$O(n \log n)$

只扫描相邻点

二维更难：相邻关系不再唯一

分治的作用，就是在二维中重新制造“少量候选”

分治思想



左半部分

递归得到 δ_L

右半部分

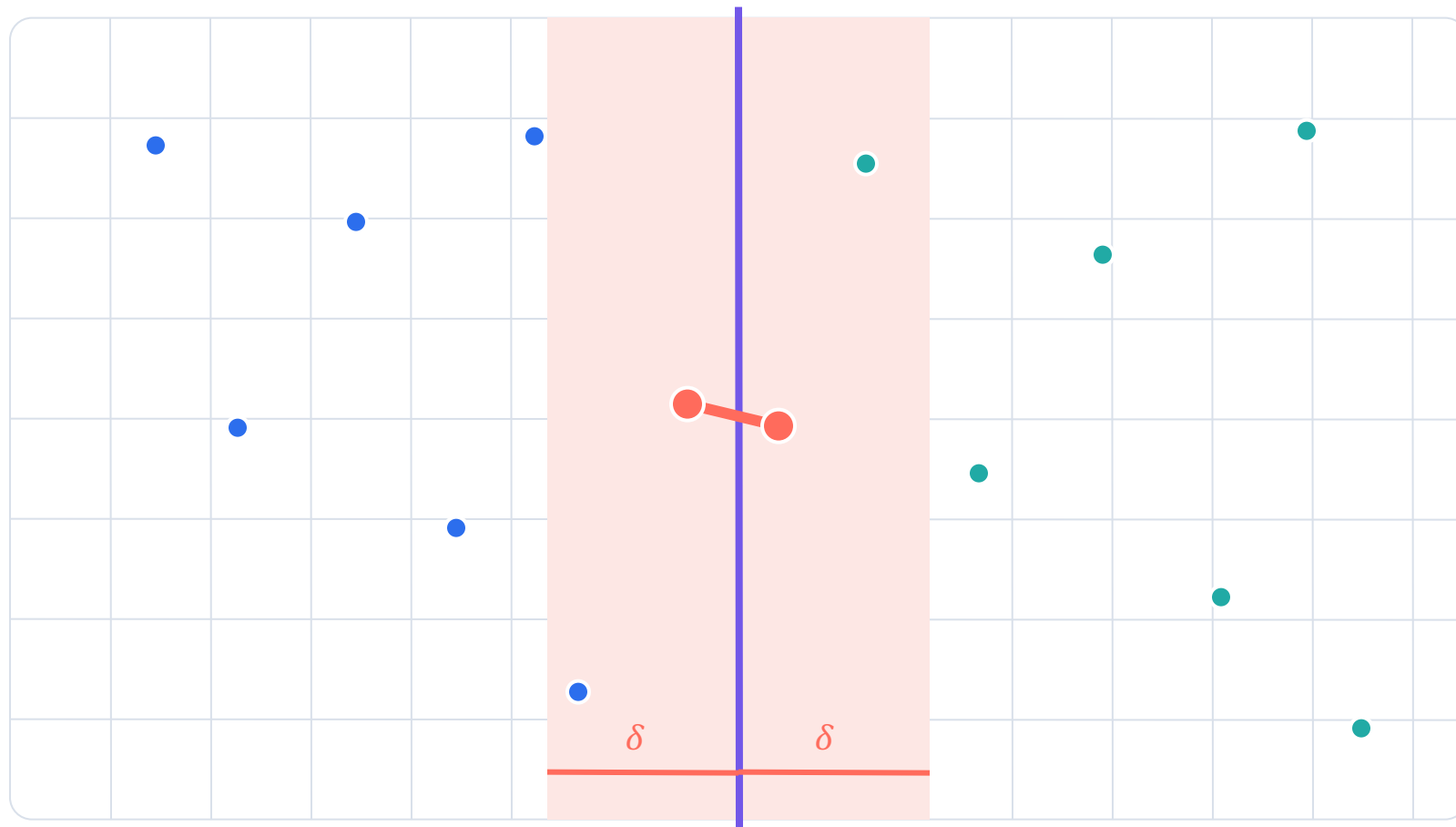
递归得到 δ_R

当前上界

$$\delta = \min(\delta_L, \delta_R)$$

还缺一步：跨越中线的点对

合并阶段的补全



若跨界点对优于 δ ，它们必定都落在中线两侧距离 δ 的条带内

为什么条带足够？

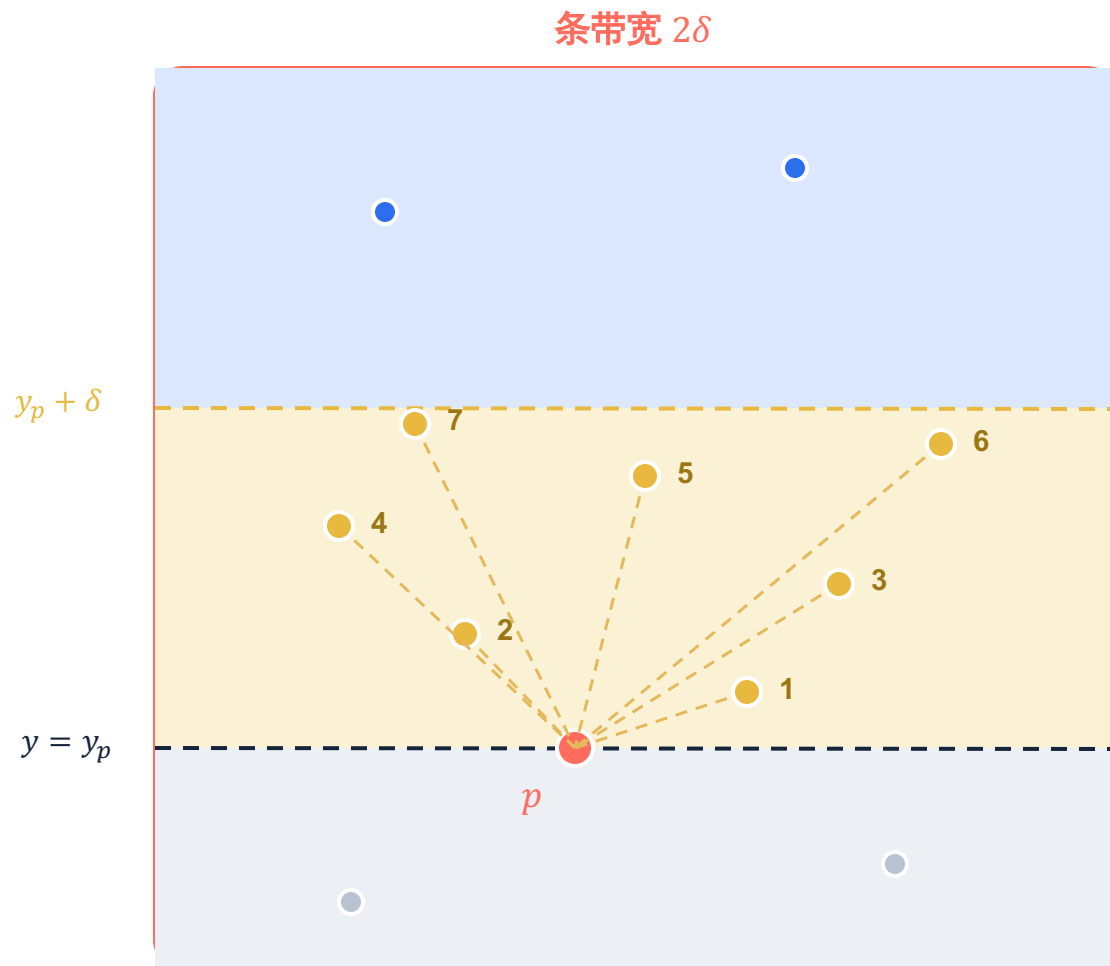
点到中线的水平距离若 $\geq \delta$
它与另一侧任意点的距离也 $\geq \delta$

候选集合

$$|x - x_{mid}| < \delta$$

条带之外直接排除

条带内按纵坐标排序



从下到上扫描，只比较纵向距离小于 δ 的后续点

扫描规则

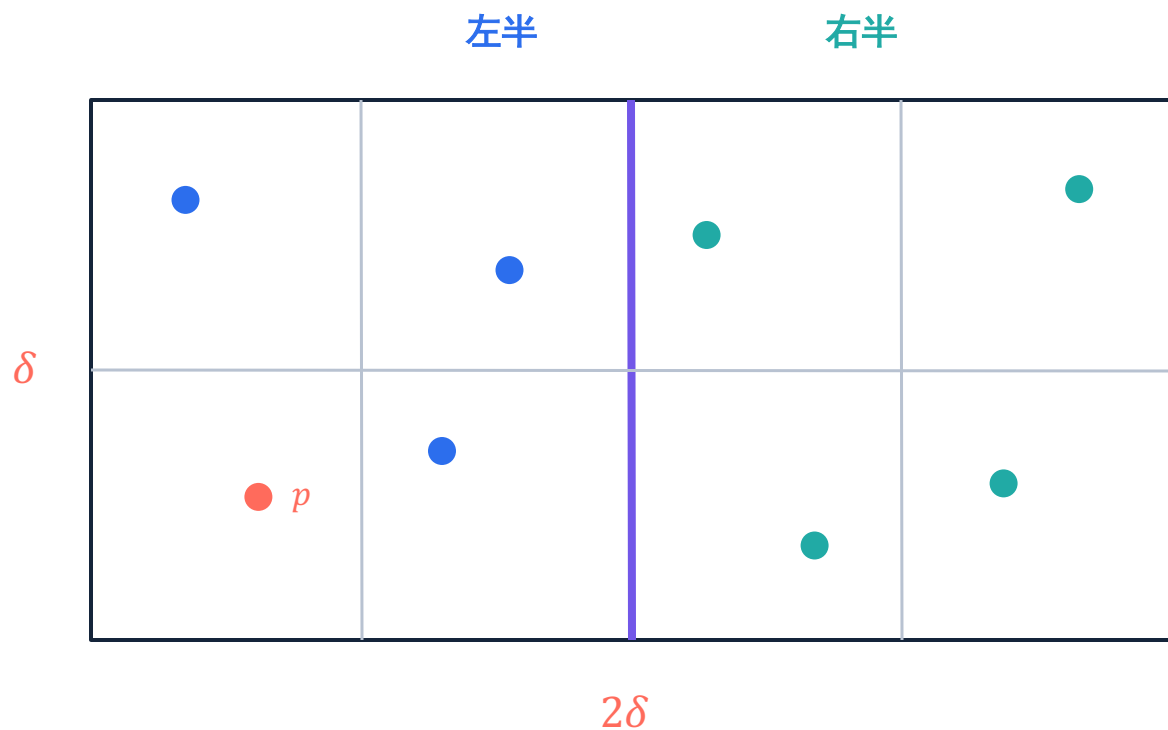
已处理 p 下方的点无需回头比较

候选 只看 $y_p < y_q < y_p + \delta$

排除 $y_q - y_p \geq \delta$ 时停止

关键结论：这个窗口里的点数有常数上界

每个窗口比较次数



几何装箱把候选数量限制为常数

把 $2\delta \times \delta$ 的窗口切成 8 个小正方形

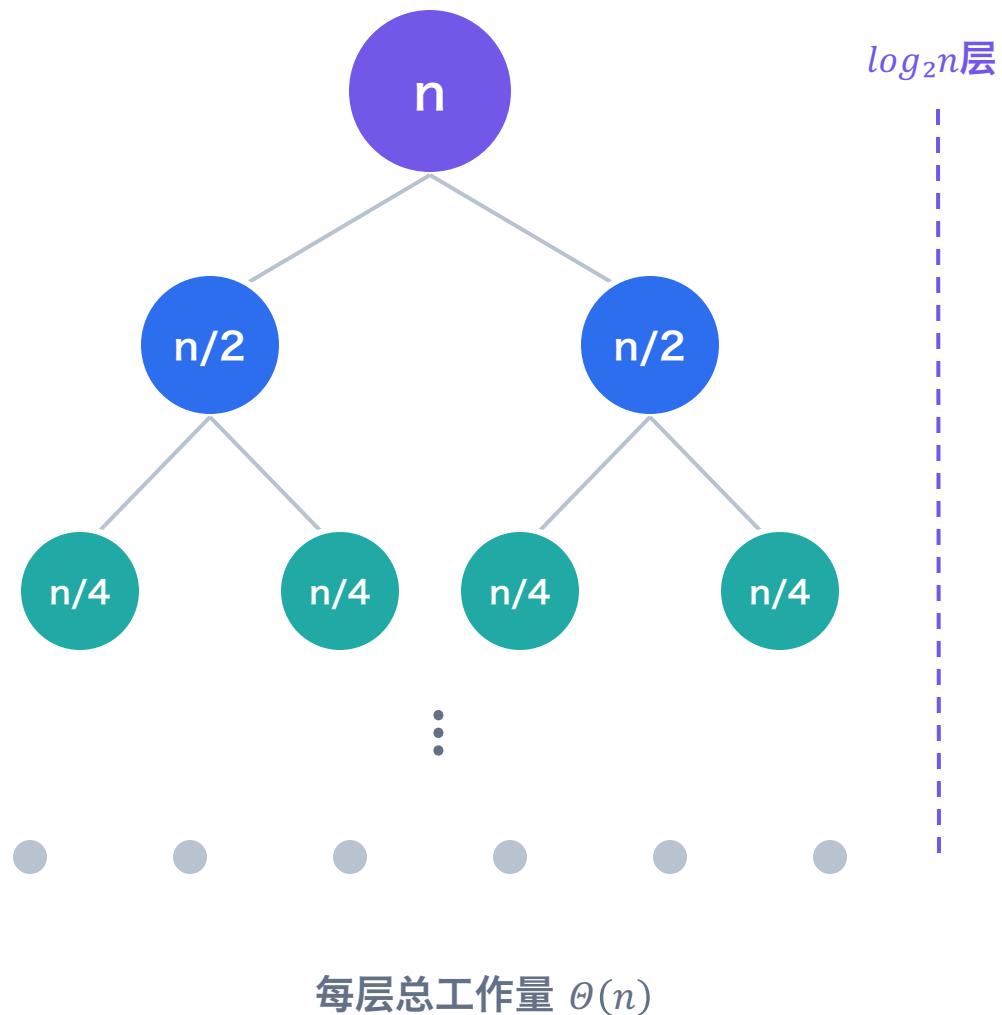
每个小格边长 $\delta/2$ ；同一小格内任意两点的距离都小于 δ 。

$$\text{小格对角线} = \delta/\sqrt{2} < \delta$$

左右两半内部都已保证最近距离 $\geq \delta$ ，因此每个小格最多 1 个点

窗口内最多 8 个点，包括 p 自身，所以 p 只需检查后续 7 个点

复杂度分析



递推式

$$T(n) = 2T(n/2) + \theta(n)$$

$$T(n) = \theta(n \log n)$$

预排序	$\theta(n \log n)$
递归合并	$\theta(n \log n)$
辅助空间	$\theta(n)$

对比暴力算法: $\theta(n^2)$

Skyline 问题定义：建筑并集的上边界

输入是矩形建筑，输出是高度发生变化的位置

输入

每栋建筑可表示为

$$B_i = [L_i, R_i, H_i]$$

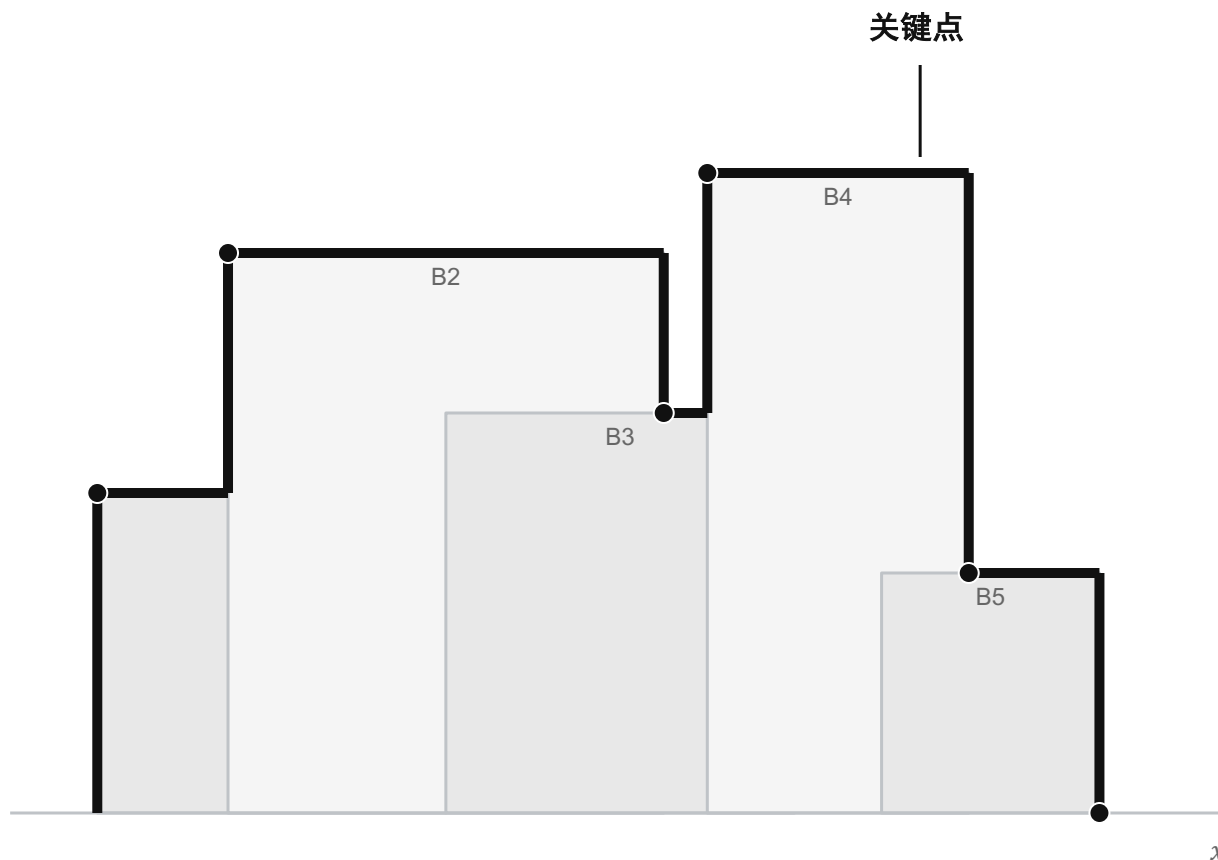
L、R 是左右边界，H 是高度

输出

按横坐标排序的关键点序列

每个关键点记录变化位置与新的可见高度

$$Skyline = [(x_1, h_1), (x_2, h_2), \dots]$$



只在最高高度改变时更新答案

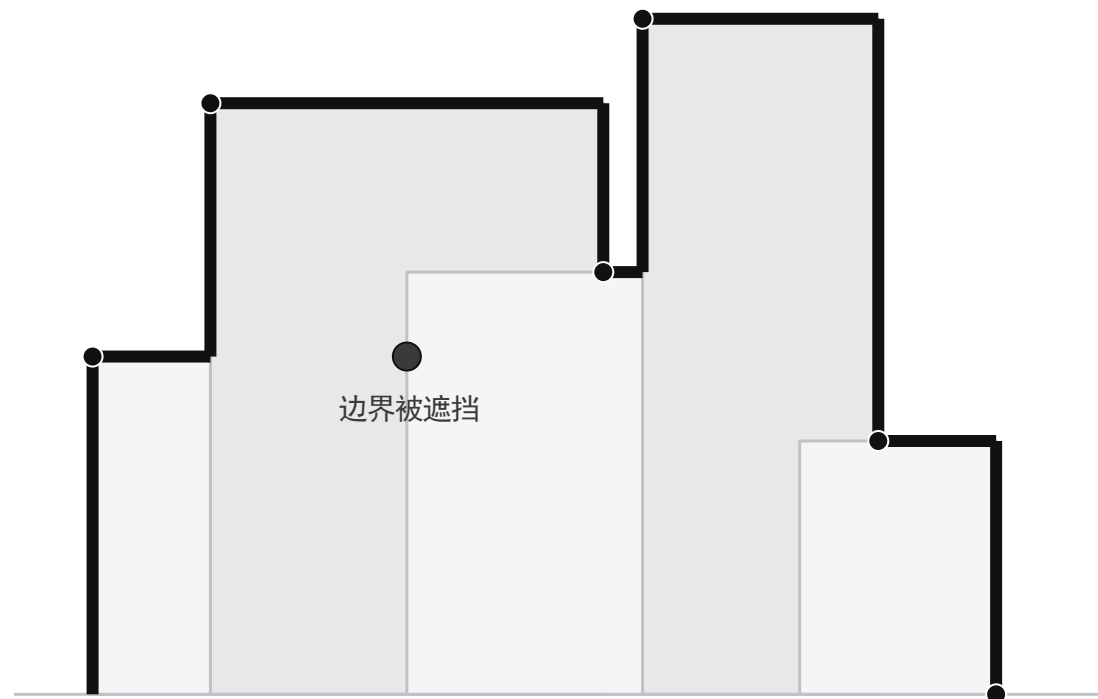
重叠把局部边界转化为一个全局最大值问题

$$H(x) = \max\{H_i | L_i \leq x < R_i\}$$

建筑边界很多，但可见轮廓只关心最上方的一条线。

被更高建筑覆盖的上升或下降，不会产生新的关键点

算法真正维护的是“当前最高高度”



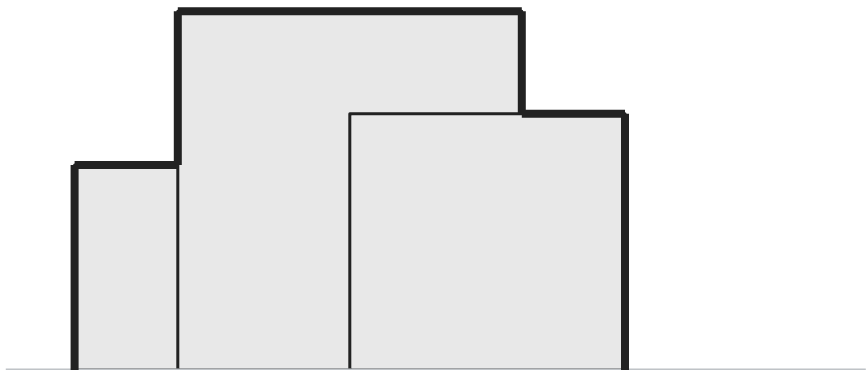
分治结构：拆建筑，合轮廓

先按左端点排序，
再把建筑列表从中间分开

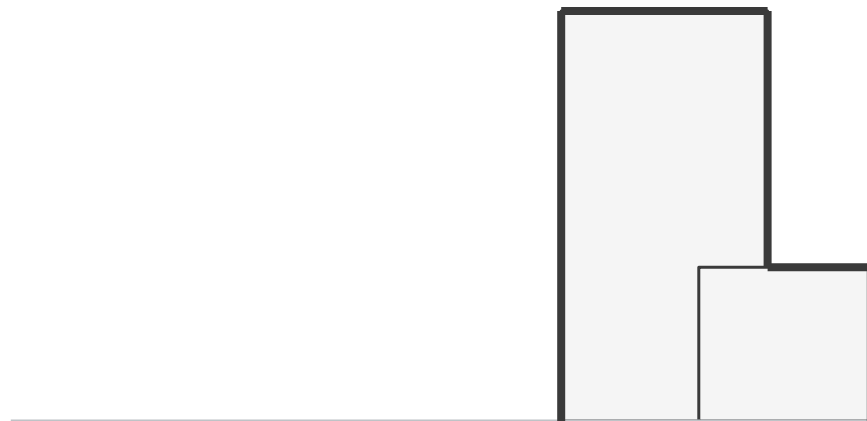
n 栋建筑

左半部分

右半部分



递归得到左天际线 L



递归得到右天际线 R

注意：两组建筑在 x 方向仍可能重叠，所以子答案不能直接拼接

子问题答案：有序关键点序列

递归基

好的中间表示让合并变得简单



$$[L, R, H] \rightarrow [(L, H), (R, 0)]$$

表示约定

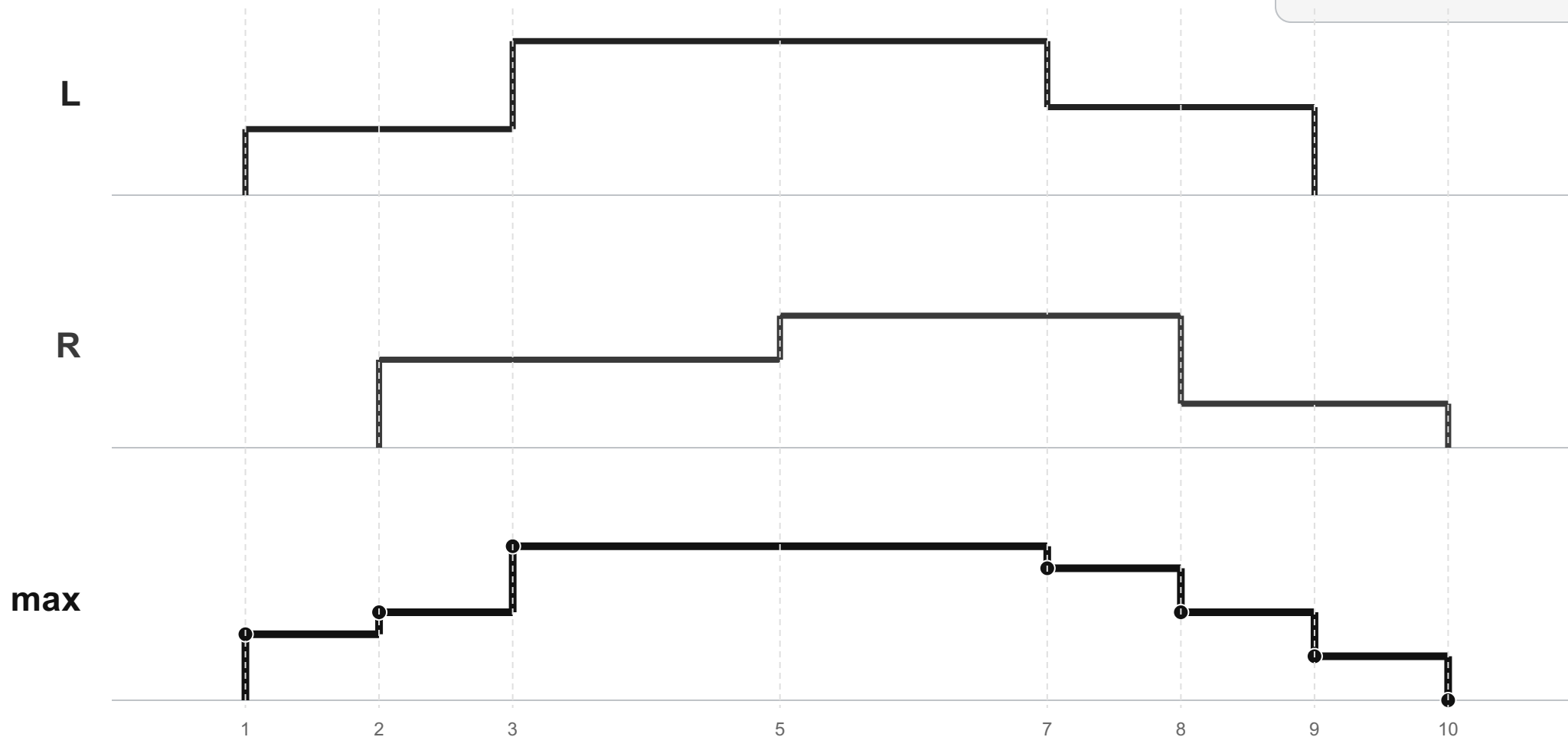
- 1 x 严格递增**
事件按从左到右排列
- 2 相邻高度不同**
轮廓不变化就不重复记录
- 3 最后回到 0**
轮廓在最右端落回地面

这三个约定同时控制正确性和输出规模

合并：同步扫描两条子天际线

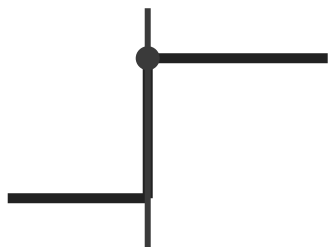
每次处理更靠左的事件，并更新对应的当前高度

$$h = \max(h_L, h_R)$$



合并结果

同一 x 有多个事件



合并后只保留一个高度

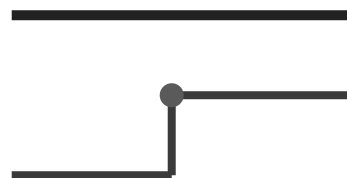
左右高度相同



$$h_L = h_R$$

最大值不变，不输出

事件被更高轮廓遮挡



子轮廓变，合并轮廓不变

相邻线段等高



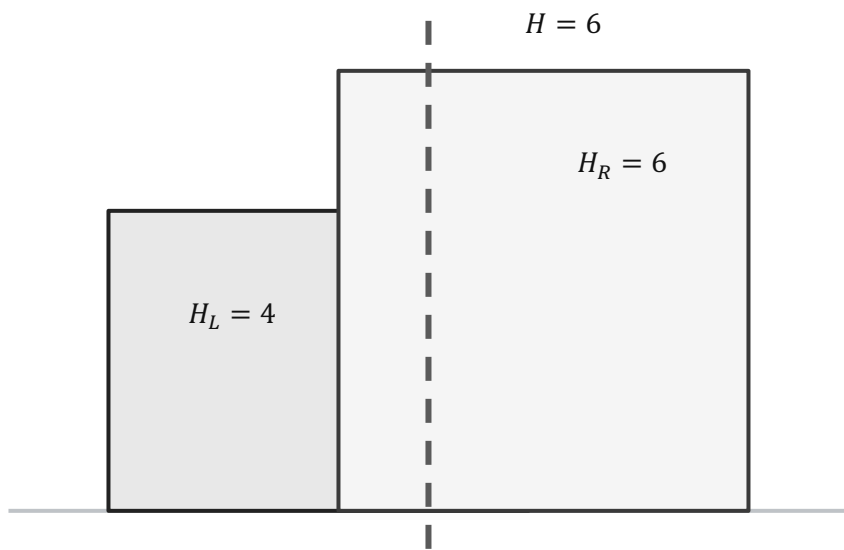
删除中间冗余点

标准化规则：同一 x 只留一个点，连续相同高度只留第一次变化

正确性分析

扫描到任意位置，记录的两个高度分别代表左右子轮廓的当前高度

$$H(x) = \max(H_L(x), H_R(x))$$



固定 x 的垂直切片

事件完整

合并轮廓的变化只可能出现在 L 或 R 的关键点处。

高度正确

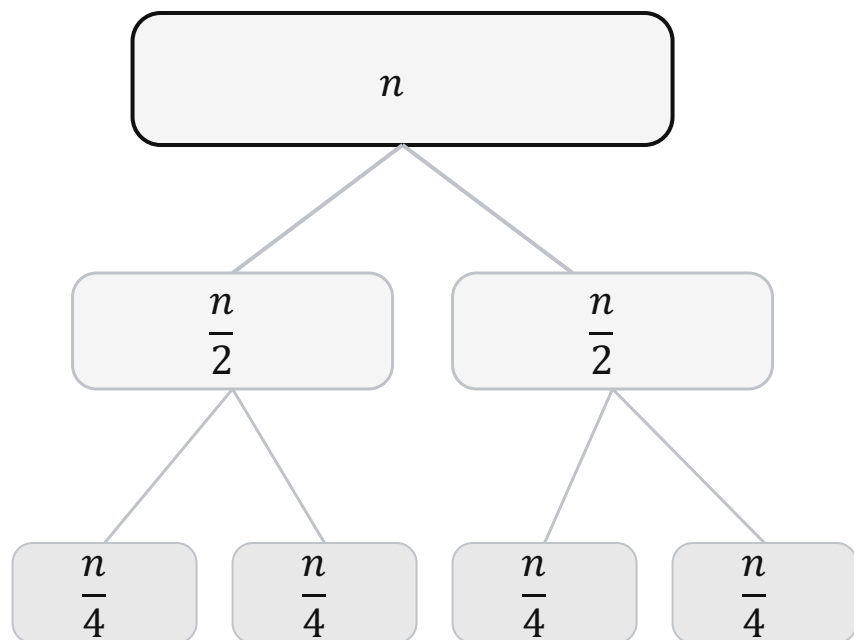
处理事件后取两个当前高度的较大值，得到这一段的真实可见高度。

结果最简

只有当最大高度改变时才输出，避免重复 x 和相邻等高点。

时间复杂度

递归树直接展示整体复杂度的来源



递归深度

$$\log_2 n$$

每层合并成本

同一层的所有子问题合计只处理线性规模的关键点

递推式

$$T(n) = 2T(n/2) + O(n)$$

时间

$$O(n \log n)$$

额外空间

$$O(n) + O(\log n)$$

中间结果与递归栈

输入若未排序，排序步骤与整体算法处于同一数量级。

从分治思想到分布式计算框架

01

递归分治



子问题在调用栈中展开
合并结果得到答案

02

单机并行



线程或进程同时执行
共享内存仍在一台机器

03

集群并行



数据与任务跨机器分布
网络和失败成为常态

04

计算框架



统一调度、通信、容错
暴露更简单的编程模型

核心变化：跨机器执行要求显式处理数据位置、网络代价和部分失败

分治思想的应用

AI 场景	怎么“分”	怎么“合”	和分治的关系
决策树	按特征把样本划成左右子集	子树组成整棵树	非常典型
ANN / KD-tree	空间递归切分	搜索候选集合	典型分治
大规模矩阵乘法	矩阵分块	块结果相加	经典分治 → AI 核心计算
分布式训练	数据拆给多 GPU	聚合梯度	广义分治
Tensor Parallel	一个矩阵拆给多 GPU	All-Reduce / All-Gather	广义分治
Mixture-of-Experts	token 分给不同专家	汇总专家输出	广义分治
大模型检索 RAG	文档分块/分区检索	汇总候选	广义分治
GPU Reduction	数据分块求局部 max/sum	树形归约	非常漂亮的分治

从单台服务器到分布式服务器集群

数据规模

内存与本地磁盘容不下完整工作集

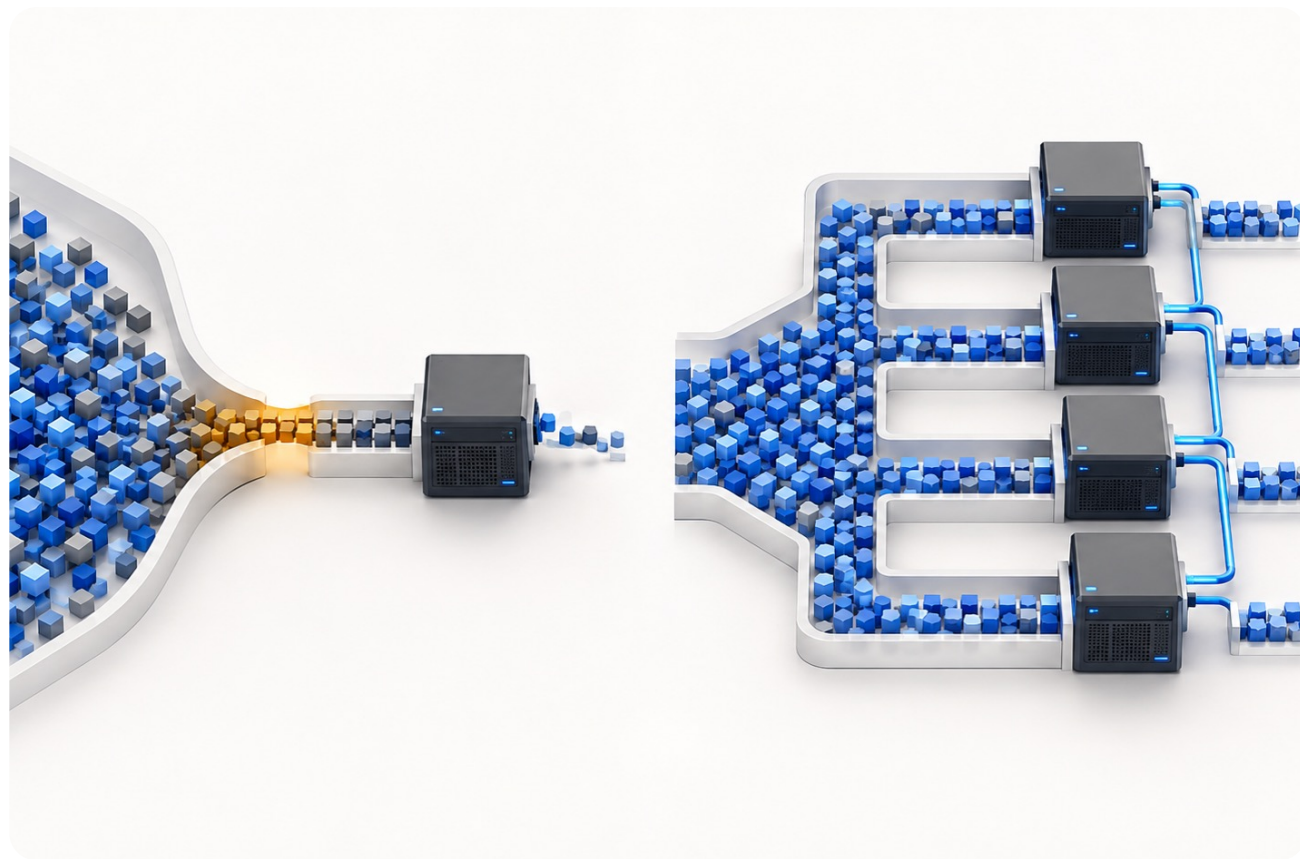
吞吐需求

单个 CPU 与 I/O 通道无法按时完成

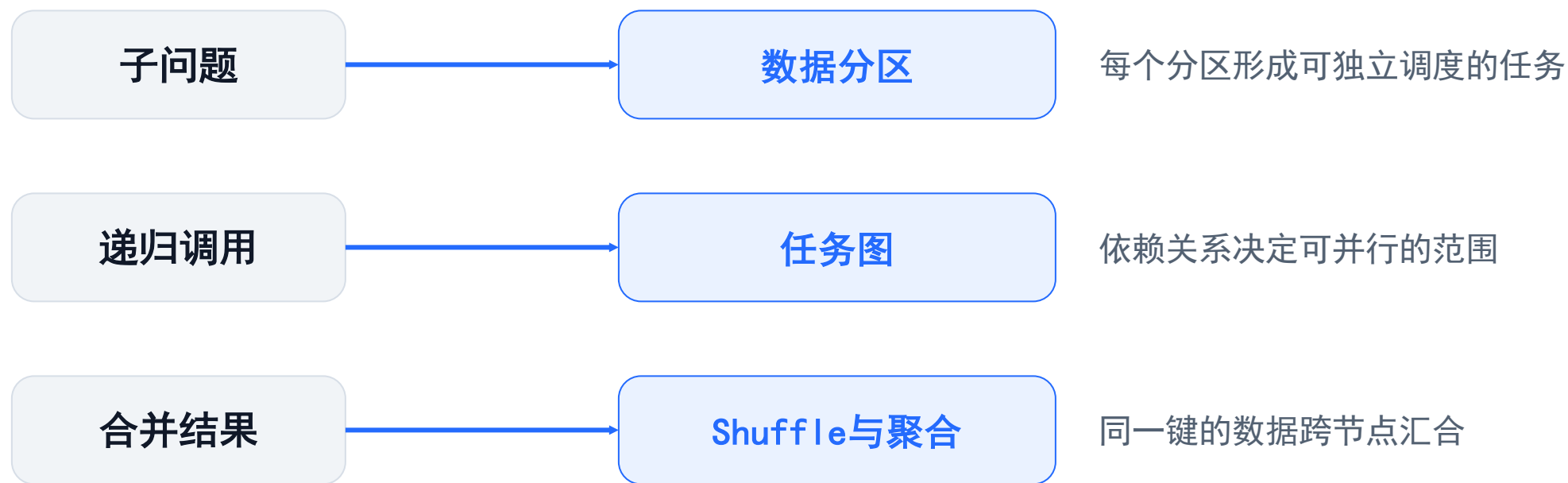
可靠性

长任务运行越久，遇到故障的概率越高

增加机器数量，让数据分区靠近计算

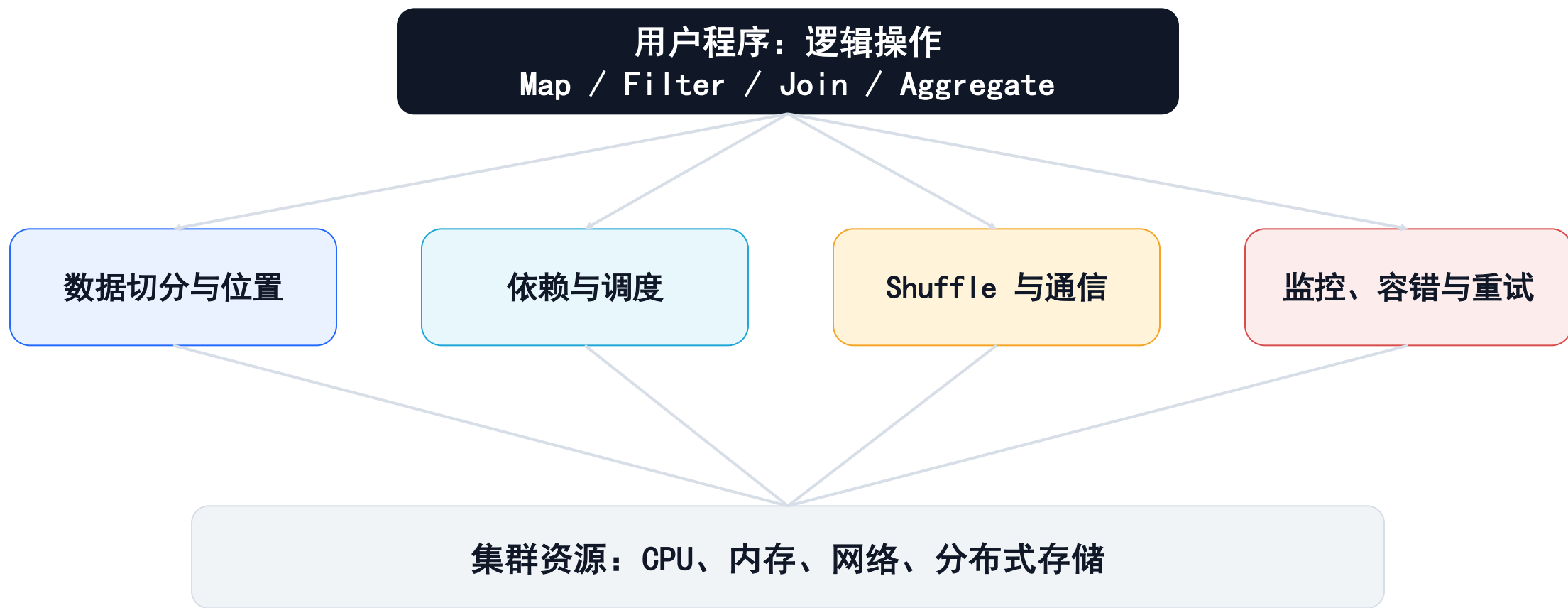


从分治到分布式



分布式系统的新问题：框架接管数据放置、任务调度、通信协调和失败恢复

分布式并行计算框架



程序员描述“算什么”，分布式框架把“逻辑计算”翻译成“集群上的实际执行”

WordCount: 数据如何流动

输入分片

spark makes
data fast
data makes
insight

Map 输出

(spark,1)
(makes,1)
(data,1)
(fast,1)
(data,1)
(makes,1)
(insight,1)

Shuffle 分组

data [1,1]
makes [1,1]
fast [1]
insight [1]
spark [1]

Reduce 输出

data 2
makes 2
fast 1
insight 1
spark 1

Map 可并行

网络重排

Reduce 可并行

可选 Combiner 可先在 Map 端局部聚合，减少传输量

文本有依赖，生成就必须从左到右吗？

- 逐 token 接龙：已有前缀 → 决定下一个位置
- 多轮填空：保留空位 → 同时考虑多个位置
- 关键问题：有依赖，是否就意味着全程串行？

逐 token 接龙

我		
我	明天	
我	明天	去

多轮填空

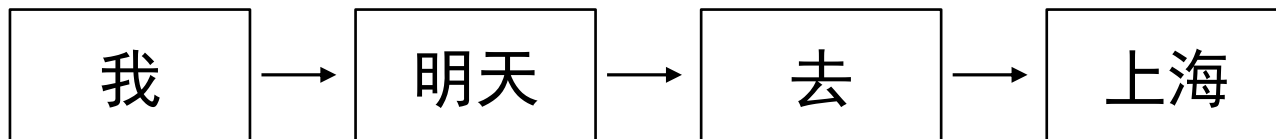
M	M	M
我	M	去
我	明天	去

M = [MASK] (掩码) 教学用 token ≠ 一个汉字，也未必等于一个词

为什么传统 LLM 通常逐 token 生成？

- 把整句概率拆成“给定前缀，预测下一个 token”

“下一个”依赖“已经选定的前缀”



- 训练目标直接；生成时可自然向后延伸

- 后一个位置需要前面已选定的结果

训练时

真实前缀已知
许多位置可并行计算

- 已完成前缀可复用计算：KV cache

生成时

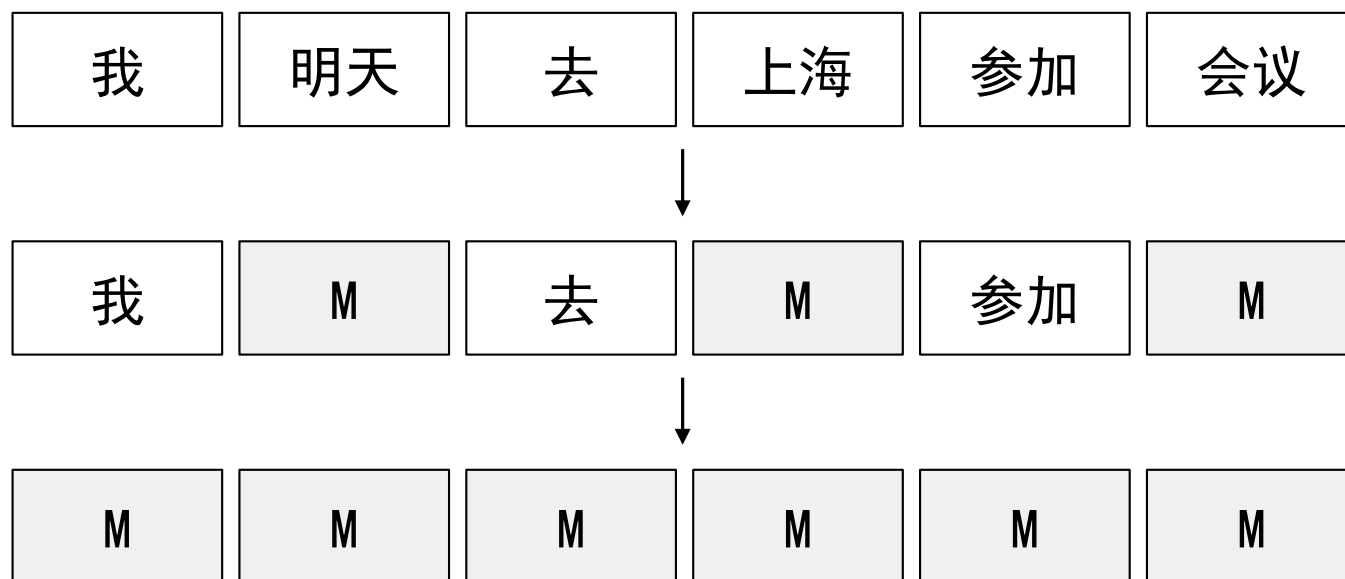
前缀需要先生成
基础解码沿 token 链推进

“逐 token”描述基础解码顺序；模型内部与训练过程仍可并行。

Diffusion 的直觉：从缺失信息到完整文本

- 加噪：逐渐遮住 token，让信息变少
- 去噪 denoising：学习依据上下文补全
- 生成：由高噪声状态，经多轮得到文本
- 聚焦 masked diffusion（掩码扩散）

前向加噪：信息逐渐减少



学习反向
多轮去噪

“填空”是直觉；生成是在构造合理文本，并非找回一条预先隐藏的答案。

训练：随机遮住，让模型学会恢复

- 取一条真实文本，随机选择掩码比例
- 把部分位置替换为 [MASK]
- 模型同时预测被遮住位置的 token
- 用原文检查预测，更新模型参数

① 真实文本

我	明天	去	上海	参加	会议
---	----	---	----	----	----

↓ 随机 masking

我	M	去	M	参加	M
---	---	---	---	----	---

↓

② 模型同时预测空位

↓

③ 与原文比较 → 更新参数

多种掩码比例，让模型练习从“少量缺失”到“几乎全空”的恢复任务。

生成：一轮预测多个位置，多轮逐步确定

- 保留用户提示，为回答准备一组 M
- 每轮依据当前整段状态，同时预测多个空位
- 按采样策略确定一部分位置，余下继续填
- 轮次结束，得到一个完整回答

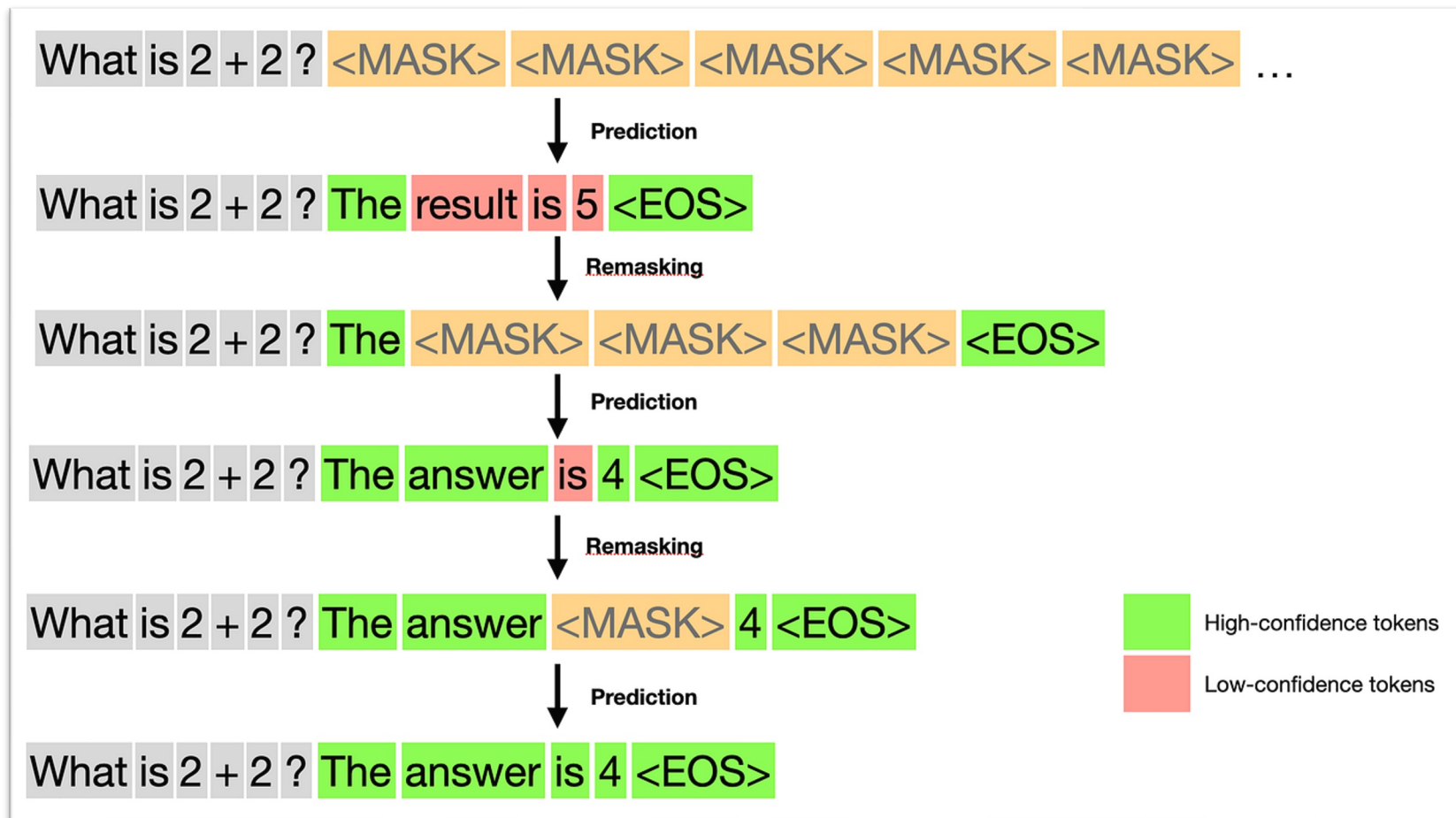
固定提示：“说说你明天的安排”

第 0 轮	M	M	M	M	M	M
第 1 轮	我	M	M	上海	M	M
第 2 轮	我	明天	M	上海	M	会议
第 3 轮	我	明天	去	上海	参加	会议

粗框：本轮新确定 | 灰底 M：仍未确定

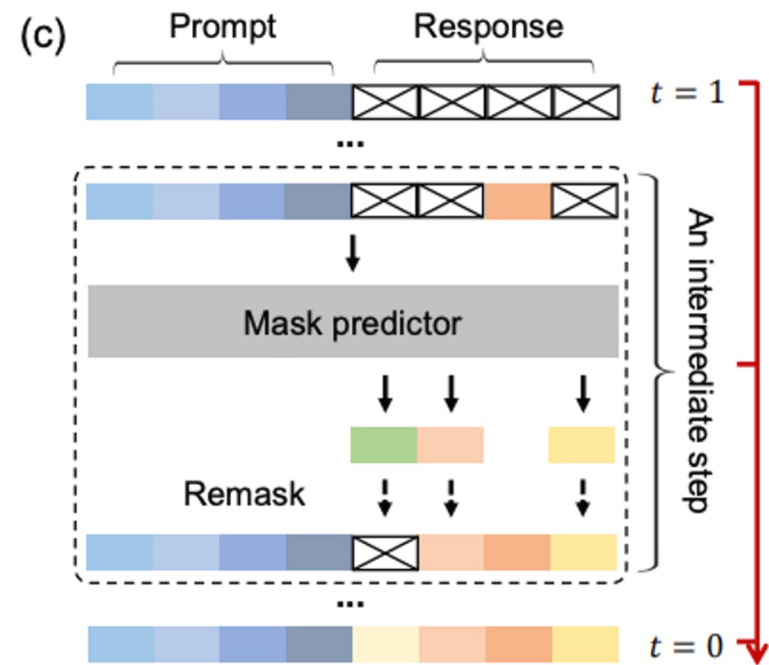
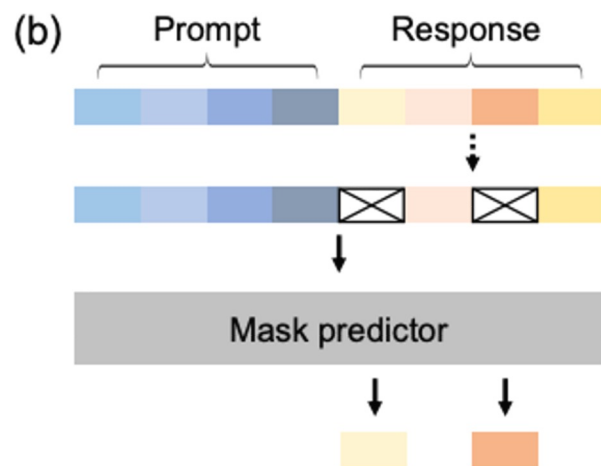
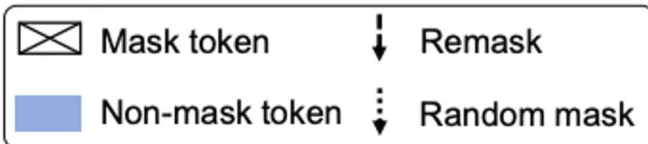
同一轮的预测共享当前上下文；新信息在后续轮次中继续发挥作用。

生成：一轮预测多个位置，多轮逐步确定



预训练, 微调, 生成

(a) Mask all tokens independently



好处和代价

好处

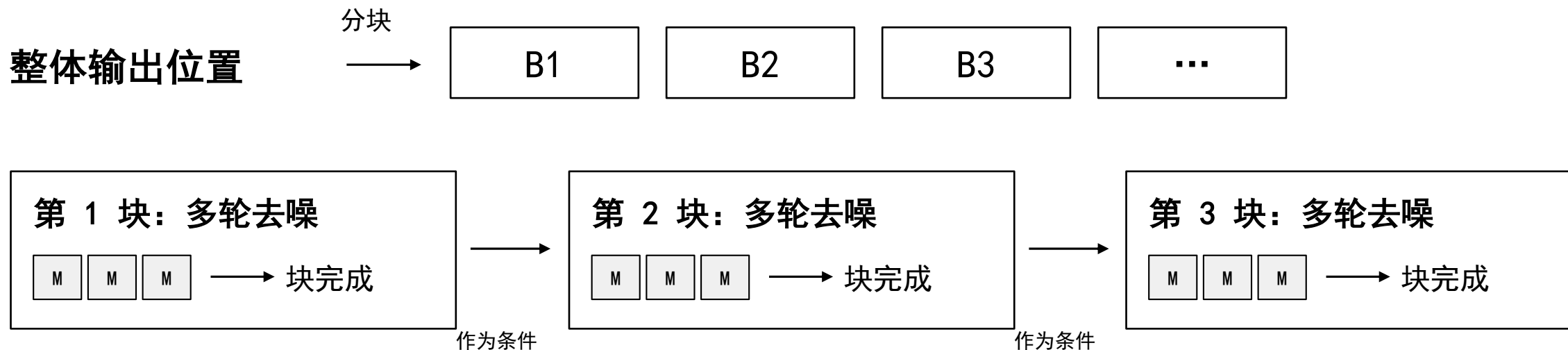
- 灵活安排位置的生成顺序
- 同轮处理多个 token 位置
- 利用左右已知信息进行补全

代价

- 固定输出长
- 每轮处理较长序列，计算与显存压力增加
- 双向状态不断变化，常规 KV cache 难直接复用

Block Diffusion: 块内并行, 块间顺序

- 把输出位置划成多个 block
- 当前块内部, 多轮并行去噪
- 前面块完成后, 作为下一块的条件
- 逐块追加输出, 支持灵活长度生成



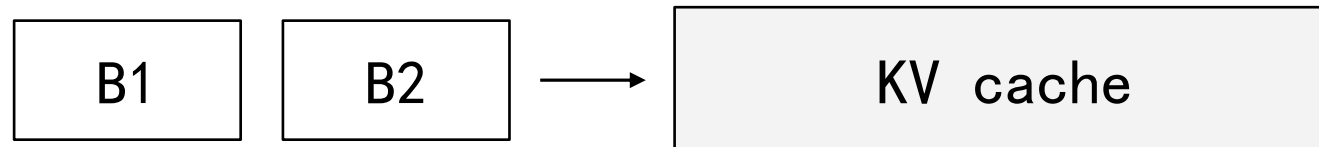
块内: 多个位置可并行预测。

块间: 下一块等待前面块完成。

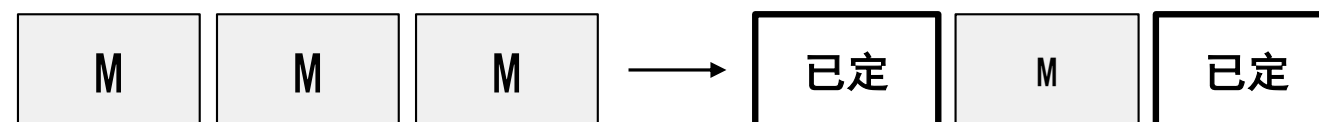
为什么分块有利于复用 **KV cache**?

- 已完成块固定，并且不读取未来块
- 前缀的注意力中间结果可缓存复用
- 当前块允许双向交互，仍需多轮计算
- 完成后加入前缀缓存，再推进下一块

已完成前缀



当前块 B3：共享固定前缀



继续去噪 → 完成 → 写入缓存

B4：尚未生成

可复用的关键：过去块不读取未来块，前缀表示保持稳定。

示例

Autoregression: ✓ High quality ✓ Arbitrary-length ✓ KV caching ✗ Not parallelizable

There

Diffusion: ✗ Lower quality ✗ Fixed-length ✗ No KV caching ✓ Parallelizable

Block Diffusion: ✓ High quality ✓ Arbitrary-length ✓ KV caching ✓ Parallelizable

练习题

- $T(n) = 4T(n/2) + n^2, T(1) = 1$
- 要求：用假设验证法猜测 $T(n)$ 的渐近时间复杂度

$$T(n) = cn^2 \log n.$$

代入：

$$\begin{aligned} 4c \left(\frac{n}{2}\right)^2 \log \frac{n}{2} + n^2 &= cn^2 (\log n - 1) + n^2 \\ &= cn^2 \log n + (1 - c)n^2 \end{aligned}$$

练习题

- $T(n) = 9T\left(\frac{n}{3}\right) + n^2 + n\log^4 n$
- 能否用主定理求解？

$a = 9, b = 3$, 所以 $n^{\log_b a} = n^{\log_3 9} = n^2$. $f(n) = n^2 + n\log^4 n$.

因为 $n\log^4 n = o(n^2)$, 所以 $f(n) = \Theta(n^2)$.

于是: $f(n) = \Theta(n^{\log_b a})$, $T(n) = \Theta(n^2 \lg n)$

属于主定理 **Case 2**: 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \lg n)$

练习题

- 假设在一场音乐会中，有位女士把她们的外套寄存在衣帽间（每个人有且只有1件外套）。然而，当演出结束时，衣帽间服务员弄丢了所有的寄存凭证，不知道哪件外套属于谁。所有女士都穿着几乎一模一样的黑色外套，但这些外套尺寸不同。服务员可以让某位女士试穿一件外套，并得到以下三种结果之一：正好合身（表示这件外套属于该女士），外套太大，外套太小。
- 但是，服务员不能直接比较两件外套的尺寸，也不能直接比较两位女士的身材大小。请给出一种算法，使服务员能够确定每件外套的归属。
- 并简要分析给出算法最坏情况下的时间复杂度。

练习题答案

- 首先选定一件外套作为“枢轴”，让所有女士试穿这件外套，并根据试穿结果将她们分为两组：一组比外套“枢轴”身材小的女士，另一组比“枢轴”大的女士。这样可以确定有一位女士的身材正好适合这件外套。接下来，让这位女士试穿其他所有外套，通过试穿判断每件外套相对她的大小。这样，在线性时间内，可以将所有外套分为两组：比这位女士“枢轴”小的外套和比她大的外套。对于每组女士和外套，递归执行相同的过程，直到所有女士和外套都匹配完成。这个算法类似于快速排序的思想，分区步骤是线性时间，且每次递归将问题规模减少一半。通过使用随机选择的“枢轴”，该算法的期望时间复杂度为 $O(n \log n)$ ，满足题目要求。
- 在每一层递归中，算法首先选定一件外套作为“枢轴”，让所有女士试穿这件外套，并根据试穿的结果将女士分为两组：一组比这件外套身材小，一组比这件外套身材大。该过程对每位女士进行一次试穿操作，耗时 $O(n)$ 。然后，确定与这件外套匹配的那位女士，接着让这位女士试穿所有其他外套。通过试穿来确定每件外套的相对大小，这同样需要对每件外套进行一次试穿操作，耗时 $O(n)$ 。因此，每一层递归的分区步骤总共耗时 $O(n)$ 。每一层递归之后，问题规模缩小为原问题的一半（类似于快速排序的分区过程）。因此，在最坏情况下，递归的深度为 $\log n$ ，因为每次递归都将问题分成两组。在每一层递归中，分区步骤的时间复杂度为 $O(n)$ ，而递归的深度为 $O(\log n)$ 。因此，总的运行时间可以表示为： $T(n) = O(n \log n)$ 。

大作业

- 发布在 elearning 上，通过 elearning 提交（有deadline）